ENHANCING SDN NETWORK SECURITY USING GAME THEORY

Prepared by

Nagham Attieh

Bayan Alnassari

Supervised by

Dr. Wassim Juneidi

Eng. Mohammad Yamen Hallak

2026

Table of Contents

List of Tables

List of Figures

Abstract

Software-Defined Networking (SDN) provides centralized and programmable network control, enabling security mechanisms to monitor traffic and apply mitigation actions dynamically. However, the centralized nature of SDN also introduces security challenges, particularly when the controller and network resources are exposed to Distributed Denial-of-Service (DDoS), flooding, spoofing, and evolving multi-stage attacks. This project presents an enhanced adaptive SDN security framework that integrates network-based intrusion detection, multi-stage attack analysis, Artificial Intelligence-based classification, Game Theory, and OpenFlow-based mitigation within a unified defense workflow.

The original phase of the project established the practical SDN security environment using Mininet, Open vSwitch, the Ryu Controller, Snort IDS, and an alert-forwarding mechanism. Network traffic is monitored and mirrored for inspection, while Snort-generated security evidence is transferred to the controller. The controller evaluates network observations and uses a dynamic Game Theory decision process to select proportional defense strategies, including ALLOW, three levels of rate limiting, and BLOCK. The selected strategy is subsequently translated into OpenFlow-compatible actions for enforcement within the SDN environment.

The extended phase enhances this framework by introducing multi-stage attack analysis and a data-driven Artificial Intelligence pipeline. Network-traffic data is cleaned, enhanced using port-variance feature engineering, standardized, and transformed using Principal Component Analysis. Attack pattern segmentation is then applied to divide the selected traffic families into detailed behavioral subpatterns. A hierarchical CatBoost classification mechanism predicts the general attack family and subsequently identifies the corresponding behavioral subpattern. Family-level and subpattern-level probabilities are combined to generate a joint confidence score, providing more informative detection evidence for the defense process.

The resulting Artificial Intelligence output is transferred through an integration bridge to the VMware-based SDN environment and incorporated into the existing stage-aware defense mechanism. Markov-based state tracking represents changes in attack behavior across successive observations, while the Game Theory engine combines multi-stage information, AI detection evidence, network observations, and defender payoffs to select an appropriate mitigation strategy. The selected defense decision is then mapped to an SDN action and converted into OpenFlow meter or flow-rule templates.

Experimental evaluation demonstrates that the enhanced framework successfully connects detailed attack identification with adaptive defense decision making. The hierarchical classifier achieved high family-level and subpattern-level accuracy, while the integration stages successfully transferred AI detection results to the VMware environment, generated Game Theory decisions, mapped them to SDN actions, and produced OpenFlow-compatible enforcement rules. Overall, the project demonstrates an integrated approach in which SDN programmability, intrusion detection, Artificial Intelligence, multi-stage analysis, and strategic defense reasoning cooperate to provide adaptive and context-aware network protection.

Keywords: Software-Defined Networking, Artificial Intelligence, Multi-Stage Attacks, Game Theory, CatBoost, Snort IDS, OpenFlow, Attack Pattern Segmentation, Adaptive Mitigation.

1 Chapter One: Introduction

1.1 Background

Software-Defined Networking (SDN) has emerged as a transformative networking paradigm that separates the control plane from the data plane and centralizes network intelligence within a programmable controller. This architectural shift has significantly improved network manageability, scalability, and flexibility, allowing administrators to configure policies, optimize traffic flows, and deploy services more efficiently than in traditional networks. As SDN continues to gain importance in cloud computing, virtualization, and security-aware infrastructures, its programmability has made it a promising platform for building more responsive and adaptive defense mechanisms [1], [2], [3].

Despite these advantages, SDN introduces new security concerns. The controller, as the logical core of the network, becomes a strategic target for adversaries. Among the most serious threats in this context are Distributed Denial-of-Service (DDoS) attacks, which attempt to overwhelm the control layer with malicious traffic, degrade network responsiveness, and disrupt service availability. The literature consistently emphasizes that the same centralization that gives SDN its strength can also become its greatest vulnerability if not properly protected [1], [3], [5].

In recent years, research has increasingly moved beyond static protection methods toward adaptive security frameworks that combine centralized control, intelligent traffic inspection, and dynamic response. This shift is particularly visible in studies that integrate SDN with Intrusion Detection Systems (IDSs), flow-based traffic monitoring, and game-theoretic decision models. These directions are highly relevant to the present project because they support the design of a framework that does not stop at attack detection, but extends toward strategic and context-aware mitigation [1], [4], [5].

Advanced cyberattacks may involve multiple attack vectors, multi-stage processes, and different penetration strategies. A major detection challenge is that some behaviors associated with these attacks may resemble legitimate network activity or may share similar statistical characteristics with other attack patterns. Consequently, Artificial Intelligence-based intrusion detection models may find it difficult to learn clear decision boundaries between normal traffic and complex attack behaviors [8].

To address this limitation, recent research has proposed attack pattern segmentation as a method for dividing broad attack behaviors into more detailed subpatterns. In this approach,

clustering is used to identify statistically distinct subpatterns within the network traffic data, after which a stacking ensemble is trained to classify and detect the segmented attack patterns [8]. This provides a more detailed representation of complex attacks than relying only on a single general label for each attack family.

Multi-stage cybersecurity can also be examined as a strategic interaction between an attacker and a defender across different stages of an attack. The Cyber Kill Chain describes the progression of a cyberattack through multiple stages, while Game Theory can be used to analyze the strategies, probabilities, and payoffs associated with attacker–defender decisions. Combining these concepts can support the evaluation of defensive strategies at different stages of an attack [9].

Based on these directions, the present project extends the original SDN security framework by adding multi-stage attack analysis and Artificial Intelligence-based detection. The existing Game Theory and OpenFlow-based mitigation mechanisms are preserved, while the detection stage is enhanced to provide more detailed information about the identified attack behavior for the subsequent defense decision process.

1.2 Problem Statement

Although SDN offers centralized control and fine-grained programmability, it also creates a critical security challenge: the controller may act as a single point of failure. When the controller is targeted by DDoS attacks or overwhelmed by abnormal flow requests, the entire network may suffer from latency, packet loss, reduced service quality, or complete disruption. This issue becomes more severe in dynamic environments where legitimate and malicious traffic must be distinguished in real time [1], [2], [3].

Existing SDN security solutions often exhibit one or more limitations. Some approaches are detection-oriented and do not provide a direct and integrated mitigation mechanism. Others achieve strong classification performance but rely on computationally expensive machine learning or deep learning techniques that may increase overhead and reduce suitability for real-time deployment near the controller. In addition, many studies are still evaluated primarily in simulated or controlled environments, with limited validation in realistic or resource-constrained settings [3], [4], [5].

Accordingly, the core problem addressed in this project is the lack of a practical, adaptive, and lightweight SDN security framework that can detect DDoS-related malicious traffic and support appropriate mitigation decisions without imposing excessive burden on the controller or the surrounding infrastructure [1], [3], [5].

In addition to the controller-related security challenges, accurately identifying multi-stage cyberattacks presents a separate detection problem. The network-flow data associated with these attacks may contain several behaviors that belong to the same attack campaign but are represented under one general class label. Some of these behaviors may also resemble benign traffic or share similar statistical distributions with other attack patterns, increasing the possibility of misclassification by machine learning-based intrusion detection systems [8].

Therefore, relying only on broad attack labels may not provide sufficient information to represent the detailed behavioral structure of a multi-stage attack. A more detailed representation is needed to distinguish the subpatterns that compose complex attack behaviors and to improve their subsequent classification and detection [8]. Moreover, because an attack develops through different stages, the defensive response should consider the changing interaction between the attacker and defender rather than treating the entire attack as one isolated event [9].

1.3 Project Motivation

The motivation for this project is both practical and academic. From a practical perspective, modern programmable networks require more than passive monitoring. A useful security framework must be capable of observing malicious behavior, interpreting it correctly, and responding with a suitable mitigation action. This is particularly important in SDN, where the centralized controller gives the network the ability to enforce policies dynamically and at scale [1], [2].

From an academic perspective, the reviewed literature reveals a persistent gap between detection and mitigation. The SEC-SDN study demonstrates the value of integrating Snort and game-theoretic reasoning into an SDN framework for adaptive DDoS defense. The hybrid mitigation study goes further by stating explicitly that detection alone is not adequate, and that

mitigation should be informed by adaptive, strategic reasoning rather than fixed thresholds alone. These findings strongly support the conceptual direction of the current project [1], [4].

The project is also motivated by the growing need for efficient and deployable solutions. Recent work on resource-aware DDoS detection in SDN has shown that high predictive performance must be balanced with memory usage, training time, inference latency, and controller stability. Therefore, a project that combines SDN programmability, practical inspection tools, and adaptive decision-making can offer both academic value and implementation relevance [3], [5].

The continuation of this project is motivated by the need to extend the original framework from the detection of individual flooding and spoofing events toward the analysis of more complex multi-stage attack behavior. The first project phase established the SDN environment, traffic inspection mechanism, game-theoretic decision engine, and OpenFlow-based mitigation process. The current phase preserves these components while enhancing the attack analysis and detection stage.

The enhanced project introduces dataset-based Artificial Intelligence processing to identify both general attack families and more detailed behavioral subpatterns. Attack pattern segmentation and ensemble learning are adopted as the initial methodological basis, while an improved hierarchical classification mechanism is developed within the project to predict the attack family first and then identify the corresponding subpattern. The resulting detection information is intended to provide a more detailed input to the existing Game Theory-based defense process.

1.4 Aim of the Project

The main aim of this project is to enhance SDN network security by designing an adaptive DDoS-oriented defense framework that integrates the Ryu Controller, OpenFlow, Open vSwitch traffic mirroring, Snort IDS, and a game-theoretic decision engine. The proposed framework focuses on detecting and mitigating distributed ICMP flooding behavior and spoofing-related traffic in an emulated SDN environment. Instead of relying on a fixed threshold, the framework evaluates multiple indicators, including packet rate, Snort alert status, packet loss, RTT, spoofing indicators, and the number of suspicious sources, in order to select a proportional mitigation action.

The extended phase of the project broadens this aim by incorporating a data-driven Artificial Intelligence pipeline for the analysis and detection of multi-stage cyberattacks. The enhanced pipeline processes network-traffic data, represents the selected features using Principal Component Analysis, segments broad attack behaviors into more detailed subpatterns, and evaluates ensemble-based classification techniques. In addition, an improved hierarchical CatBoost classifier is developed to predict the general attack family first and then identify the corresponding behavioral subpattern.

The hierarchical detection output includes the predicted attack family, the identified subpattern, and a joint confidence score calculated from the family-level and subpattern-level prediction probabilities. This detailed output is transferred through a detection–game integration bridge to support the existing stage-aware, Game Theory-based defense process. In this way, the extended framework aims to connect detailed AI-based attack identification with adaptive defense selection and OpenFlow-based mitigation.

1.5 Project Objectives

The project seeks to achieve the following objectives:

1. To examine the security weaknesses of SDN environments, especially controller exposure to DDoS-related traffic.

2. To build an emulated SDN topology using Mininet with two OpenFlow switches and multiple hosts.

3. To configure Open vSwitch mirroring so that traffic from each switch can be inspected by a dedicated Snort VM.

4. To use Snort IDS to detect suspicious ICMP flooding and spoofing-related traffic patterns.

5. To implement an alert forwarding agent that transfers normalized Snort alerts from the Snort VM to the Mininet/Ryu controller VM.

6. To design a game-theoretic decision engine that evaluates observed traffic behavior and selects one of the following actions: ALLOW, RL_1, RL_2, RL_3, or BLOCK.

7. To apply mitigation through OpenFlow-based rate limiting and drop rules.

8.To evaluate the proposed framework using normal traffic, single-source ICMP flood traffic, multi-source DDoS-oriented ICMP flood scenarios, and spoofed ICMP traffic, considering indicators such as packet rate, Snort alerts, attacker count, RTT, packet loss, and mitigation response.

9. To prepare and preprocess the selected multi-stage network-traffic dataset by selecting the target attack families, cleaning invalid values, and organizing the data for Artificial Intelligence-based analysis.

10. To transform the cleaned numerical features using Principal Component Analysis and produce a reduced feature representation suitable for the subsequent segmentation and classification stages.

11. To segment broad attack behaviors into detailed behavioral subpatterns using clustering-based attack pattern segmentation and to validate the quality of the resulting subpatterns.

12. To train and evaluate a stacking ensemble framework for the classification of the segmented multi-stage attack patterns.

13. To develop an improved hierarchical CatBoost classification mechanism that first predicts the general attack family and then applies the corresponding family-specific model to identify the final subpattern.

14. To calculate a joint confidence score by combining the family-level and subpattern-level prediction probabilities, thereby providing a more informative confidence measure for the final hierarchical prediction.

15. To model and track changes in attack behavior using Markov-based stage-transition logic and to associate the detected behavior with the progression of a multi-stage attack.

16. To transfer the Artificial Intelligence detection output, including the attack family, subpattern, and joint confidence score, to the Game Theory-based defense process through the detection–game integration bridge.

17. To evaluate the developed Artificial Intelligence models using accuracy, precision, recall, F1 score, classification reports, confusion matrices, and confidence-related analysis.

1.6 Literature Basis

The project is supported by a set of complementary studies that shape its conceptual and methodological direction.

The primary and most directly relevant study is the SEC-SDN work by Razvan and Mitica. This study proposes a Security-Enhanced SDN framework integrated with Snort and game-theoretic modeling to improve resilience against DDoS attacks. Its direct relevance to the current project lies in the fact that it combines centralized SDN control, intrusion detection, strategic defense logic, and adaptive mitigation within one integrated security framework. It therefore serves as the strongest conceptual foundation for the present work [1].

The study by Gupta, Tanwar, and Badotra is highly relevant from the practical controller-security perspective. It proposes an early DDoS detection model for SDN using sFlow-based monitoring, integrates with major SDN controllers, and evaluates detection using Mininet-based scenarios and metrics such as detection time, RTT, and packet loss. This makes it particularly valuable in supporting the controller-oriented and implementation-aware side of the project [2].

The systematic review by Wainwright et al. plays a central role in supporting the project's research justification. It shows that although many SDN-based DDoS solutions have been proposed, the literature remains dominated by simulation-heavy validation, and mitigation is rarely evaluated together with detection. It also highlights the importance of lightweight, edge-aware, and resource-conscious approaches, especially in constrained environments [3].

The hybrid approach by Kachavimath and Narayan provides strong support for the project's adaptive security direction. It argues that conventional fixed-rule approaches are insufficient for dynamic SDN traffic, and it presents a framework that combines feature selection, unsupervised clustering, posterior-probability reputation scoring, and game-theoretic mitigation. This study is especially useful because it directly supports the idea of linking detection to adaptive defense rather than treating them as separate stages [4].

The resource-aware ensemble framework by Trigui et al. contributes a modern perspective on SDN security design. It emphasizes that successful detection systems should be not only accurate but also efficient, generalizable, and feasible for deployment. Its focus on resource profiling, latency, memory usage, and controller-adjacent deployment strongly reinforces the need for practical and lightweight design decisions in the present project [5].

The study by Javadpour et al. provides secondary support for the game-theoretic dimension of the project. Although its primary focus is malicious-node and botnet-oriented modeling rather than controller flooding specifically, it demonstrates that strategic interaction between attackers and defenders can be modeled in a meaningful way in network security settings, and that SDN environments can benefit from such structured adversarial reasoning [6].

The Advanced EDoS Eye study by Chowdhury et al. offers peripheral but still useful support to the project. It is not an SDN-DDoS study in the strict sense, since it focuses on EDoS in cloud systems, but it is important because it argues that static games are impractical for real attacker–defender conflicts and that dynamic game-based decision modules can improve threshold selection, response quality, and QoS preservation. This supports the broader theoretical argument in favor of dynamic decision-making [7].

The study by Gong, Cho, and Choi provides the primary methodological basis for the Artificial Intelligence-based multi-stage attack detection component introduced in the extended project. The authors explain that advanced attacks may employ multiple attack vectors, stages, and penetration strategies, while some of their behaviors may resemble legitimate network activity or other attack patterns. To address this classification difficulty, the study proposes an attack pattern segmentation method that applies clustering to divide broad attack behaviors into detailed subpatterns. A stacking ensemble is then used to classify and detect the segmented attack patterns. The approach was evaluated using the CIC-IDS-2018 dataset and achieved an average F1-score of 0.9732 for multi-stage attack patterns [8]. Finally, the study by Kour, Karim, and Dersin strengthens the theoretical basis for analyzing cybersecurity as a strategic interaction that develops across multiple attack stages. The study integrates Game Theory with the Cyber Kill Chain, which describes the progression of a cyberattack from reconnaissance through the subsequent attack stages. It models the attacker and defender as competing players who select strategies, evaluate probabilities, and obtain payoffs according to their chosen actions. The proposed approach uses a non-cooperative mixed-strategy game and demonstrates its application through a cybersecurity case study. Its

relevance to the present project lies in supporting attacker–defender modeling, stage-related attack analysis, and strategic defense evaluation [9].

The adopted studies support the framework from complementary perspectives, including SDN-based DDoS detection, adaptive mitigation, resource-aware design, game-theoretic decision making, multi-stage attack analysis, attack pattern segmentation, and Artificial Intelligence-based classification, as summarized in Table 1.1.

Table 1.1 .Summary of the adopted studies supporting the project

Ref.	Study title	Year	Main focus	Contribution to the project
[1]	Enhancing network security through integration of game theory in software-defined networking framework	2025	SEC-SDN framework integrating SDN, Snort, and game theory for adaptive DDoS mitigation	The closest and primary reference for the proposed project design
[2]	A novel sFlow based DDoS detection model in software defined networking	2024	Early DDoS detection in SDN controllers using sFlow with practical controller-based evaluation	Supports the practical monitoring and SDN-controller detection perspective
[3]	Software-Defined Networking Security Detection Strategies and Their Limitations with a Focus on Distributed Denial-of-Service for Small to Medium-Sized Enterprises	2025	Systematic review of SDN-based DDoS detection and mitigation with emphasis on lightweight solutions	Strongly supports the research gap and the need for practical integrated solutions
[4]	Hybrid Approach for Detection and Mitigation of DDoS Attacks Using Multi-feature Selection, Unsupervised Learning, and Game Theory	2025	Adaptive detection and mitigation using feature selection, clustering, and game theory	Supports the detect-to-mitigate logic and adaptive strategic defense
[5]	Resource-Aware Ensemble Framework for DDoS Detection in SDN via Novel Game-Theoretic Feature Selection	2026	Resource-aware SDN DDoS detection with game-theoretic feature selection	Supports efficiency, deployment feasibility, and low-overhead design
[6]	Detecting malicious nodes using game theory and reinforcement learning in software-defined networks	2025	Game-theoretic and learning-based modeling of adversarial network behavior	Supports attacker–defender modeling and adaptive reasoning in SDN security

Ref.	Study title	Year	Main focus	Contribution to the project
[7]	Advanced EDoS eye: A pragmatic model to mitigate economic denial of sustainability attack in cloud system applying dynamic game-based decision module	2026	Dynamic game-based decision module for adaptive defense and threshold selection	Supports the theoretical argument for dynamic game-driven response logic
[8]	Detection of Multi-Stage Attacks Through Attack Pattern Segmentation	2025	Clustering-based attack pattern segmentation and stacking ensemble classification for multi-stage attacks	Provides the methodological basis for subpattern generation, segmented attack classification, and the Artificial Intelligence detection pipeline
[9]	Modelling Cybersecurity Strategies with Game Theory and Cyber Kill Chain	2025	Non-cooperative mixed-strategy modeling of attacker–defender interaction across Cyber Kill Chain stages	Supports attacker–defender strategy modeling, stage-related attack analysis, payoff evaluation, and strategic defense selection

1.7 Research Gap

The reviewed literature reveals several important gaps that justify the present project.

First, many SDN-DDoS solutions are still validated mainly in simulations or controlled testbeds. While such environments are useful for initial evaluation, they do not fully address deployment realism, controller stability, or operational trade-offs in practical settings. This creates uncertainty regarding how well many proposed methods would perform under realistic constraints [3].

Second, much of the literature emphasizes attack detection more than end-to-end mitigation. Some studies achieve promising results in identifying malicious traffic, but the integration between inspection, controller decision-making, and active mitigation remains limited. The lack of tightly coupled detect-to-mitigate pipelines represents a meaningful gap, particularly for SDN where the controller is capable of acting immediately after detection [3], [4].

Third, several modern detection approaches rely on computationally expensive architectures. Although these methods may produce excellent classification results, they are not always suitable for controller-adjacent deployment where latency, memory, and inference efficiency are critical. This highlights the need for security designs that combine intelligence with operational feasibility [3], [5].

Therefore, the present project addresses a clear gap by aiming to build a framework that combines SDN-based control, practical traffic inspection, and game-theoretic mitigation logic in a form that is both conceptually integrated and implementable in an academic test environment [1], [4], [5].

Based on this gap, the present project focuses on building a closed-loop experimental workflow in which mirrored traffic is inspected by Snort, alerts are forwarded to the Ryu controller, the game-theoretic engine evaluates the attack context, and OpenFlow rules are installed to apply graduated mitigation. This makes the project different from purely detection-oriented approaches because it connects observation, detection, decision-making, and mitigation in one practical SDN workflow.

An additional gap concerns the representation and detection of multi-stage attack behavior. Many intrusion detection datasets assign one broad label to network flows associated with an attack category, even though the underlying traffic may contain several statistically distinct behaviors. This broad representation can conceal meaningful subpatterns, particularly when some attack behaviors resemble benign traffic or overlap with patterns observed in other attacks. As a result, Artificial Intelligence models may experience difficulty in learning clear distinctions among complex attack behaviors [8].

Although attack pattern segmentation and stacking ensembles provide a useful basis for dividing broad attack categories into detailed subpatterns and classifying them, the reviewed study does not address their integration with an operational SDN defense framework [8]. In particular, it does not connect the detailed classification output to controller-side stage tracking, game-theoretic defense selection, or OpenFlow-based enforcement. Conversely, the existing SDN and Game Theory studies mainly emphasize attack detection, strategic response, and mitigation, but do not provide a detailed hierarchical mechanism for identifying both the general attack family and its internal behavioral subpattern.

The present project addresses this combined gap by extending the existing closed-loop SDN defense workflow with an Artificial Intelligence-based multi-stage attack analysis pipeline. The developed pipeline includes data preprocessing, feature transformation, attack pattern segmentation, ensemble classification, and an improved hierarchical CatBoost mechanism that predicts the attack family and the corresponding subpattern. A joint confidence score is then produced and transferred through the detection–game integration bridge, allowing the detailed detection output to support the existing Markov-based stage-tracking and Game Theory-based defense processes.

1.8 Challenges

The project is associated with several technical and research challenges.

One major challenge is processing latency. Because SDN depends on timely controller decisions, any delay introduced by inspection, analysis, or mitigation logic may reduce the effectiveness of the defense mechanism. A slow response can allow malicious traffic to influence the network before countermeasures take effect [5].

Another challenge is false positives. IDS-based or anomaly-based systems may occasionally classify legitimate traffic as malicious, especially when network traffic is bursty or highly dynamic. In practical environments, excessive false alarms may result in unnecessary blocking, degraded quality of service, and reduced trust in the defense mechanism [1], [3].

A third challenge is resource overhead. Controller-adjacent security systems must be designed carefully so that they do not consume excessive CPU, memory, or bandwidth. Recent literature shows that even highly accurate systems may be impractical if they impose unacceptable overhead on real-time SDN operations [3], [5].

A fourth challenge is dynamic strategy selection. Detecting suspicious traffic is only the first stage. The framework must still choose how to respond, whether by blocking, rate limiting, rerouting, or continued observation. This decision cannot always rely on static thresholds alone, which is why game theory becomes especially valuable but also intellectually demanding in this project [4], [7].

A fifth challenge is the accurate representation of multi-stage attack behavior. The traffic associated with one attack family may include several internal behaviors, while some of these behaviors may resemble benign traffic or overlap with other attack patterns. This similarity makes it difficult to generate clearly separated subpatterns and may affect the performance of the subsequent Artificial Intelligence classifiers [8].

A sixth challenge is class imbalance and the presence of rare subpatterns within the segmented dataset. Some attack behaviors may contain substantially fewer observations than others, which can reduce the reliability of model training and evaluation. Therefore, the implemented data-processing pipeline must preserve the meaningful attack classes while filtering insufficiently represented subpatterns and applying suitable balancing strategies where necessary.

A seventh challenge is maintaining consistency across the hierarchical prediction stages. An incorrect family-level prediction may direct the traffic sample to an unsuitable family-specific subpattern model. For this reason, the developed classifier evaluates family-level confidence and combines the family and subpattern probabilities into a joint confidence score. This creates a more informative final output, but also introduces the need to evaluate confidence behavior in addition to conventional classification accuracy.
Finally, integrating the Artificial Intelligence detection output with the existing defense framework represents an engineering challenge. The attack family, subpattern, and confidence information must be transferred in a consistent format through the detection–game bridge without disrupting the existing Markov-based stage-tracking, Game Theory decision logic, or OpenFlow enforcement process.

1.9 Practical Applications

The proposed project has practical relevance in both academic and operational contexts.

In enterprise and cloud-oriented SDN environments, an adaptive framework that links traffic inspection with controller-based response may improve resilience against DDoS attacks by reducing reaction time and helping preserve service continuity. Such integration can contribute to maintaining network availability and protecting the controller from overload under hostile traffic conditions [1], [5].

The project also has strong educational and experimental value. Because it can be developed in an emulated SDN environment using tools such as Mininet, Ryu, Snort, and attack-generation utilities, it offers a practical platform for studying SDN security, IDS integration, DDoS response logic, and attacker–defender strategy modelling [1], [2], [4].

Although the project is implemented in an emulated academic environment rather than a production network, it provides a practical proof-of-concept for studying how SDN programmability, IDS inspection, and game-theoretic reasoning can be combined to support adaptive security decisions.

The extended Artificial Intelligence component broadens these applications by supporting more detailed analysis of complex network attacks. Instead of producing only a general malicious or benign decision, the developed pipeline can identify the predicted attack family, its behavioral subpattern, and the associated confidence level. Such information can assist an SDN defense system in distinguishing between different forms and stages of suspicious behavior before selecting a proportional response.

The project also provides a reusable experimental platform for studying multi-stage attack datasets, attack pattern segmentation, ensemble classification, hierarchical learning, and the integration of data-driven detection with programmable network defense. This makes the framework relevant for academic laboratories, cybersecurity training, and future research that evaluates how detailed Artificial Intelligence outputs can support adaptive attacker–defender decision making.

1.10 Chapter Summary

This chapter introduced the background and significance of the project and explained why security remains a critical issue in modern Software-Defined Networks. It clarified the exposure of the centralized controller to DDoS, flooding, and spoofing-related threats and justified the need for adaptive security mechanisms that connect traffic observation, intrusion detection, strategic decision making, and programmable mitigation.

The chapter also introduced the extended direction of the project, which focuses on multi-stage attack analysis and Artificial Intelligence-based detection. It explained the difficulty of representing complex attack behavior through broad labels, the role of attack pattern

segmentation and ensemble classification, and the need for more detailed identification of attack families and their behavioral subpatterns. In addition, it presented the theoretical relationship between multi-stage attack progression, Markov-based stage tracking, attacker–defender modeling, and Game Theory-based defense selection.

The adopted literature was reviewed from complementary perspectives, including SDN-based DDoS protection, resource-aware detection, adaptive mitigation, attack pattern segmentation, multi-stage attack classification, and Game Theory with the Cyber Kill Chain. The chapter further identified the combined research gap addressed by the project, summarized its original and extended objectives, discussed the main implementation and Artificial Intelligence challenges, and highlighted the practical and academic applications of the developed framework.

2 Chapter Two: Theoretical Framework

This chapter presents the theoretical foundation of the original and extended phases of the project. The original phase is summarized through the main concepts required to understand the implemented SDN security framework, including centralized control, OpenFlow-based enforcement, intrusion detection, and Game Theory-based defense selection.

The chapter then focuses on the concepts supporting the current phase, including multi-stage cyberattacks, Markov-based stage modeling, network-traffic feature engineering, Principal Component Analysis, attack pattern segmentation, ensemble learning, hierarchical classification, and prediction-confidence analysis. These concepts provide the theoretical basis for detailed attack identification and its integration with the existing SDN defense workflow.

2.1 Software-Defined Networking and Security Architecture

Software-Defined Networking (SDN) separates the network control plane from the data plane and places the main control logic within a centralized, programmable controller. The controller manages traffic flows and network policies, while the forwarding devices execute the installed rules. This separation enables network behavior to be modified dynamically and provides the programmability required for adaptive security mechanisms [1], [2].

The SDN architecture is commonly organized into three layers: the application layer, the control layer, and the infrastructure layer. The application layer contains monitoring, policy, and security services. The control layer contains the SDN controller, which evaluates network information and determines the required actions. The infrastructure layer consists of programmable switches and hosts that forward traffic according to the controller-installed rules [1], [2].

In the implemented framework, this architecture separates the main security responsibilities while allowing them to operate as one coordinated workflow. Traffic is forwarded and mirrored by Open vSwitch in the infrastructure layer, while Snort performs traffic inspection and the Ryu controller manages analysis, decision making, and mitigation. Although centralized control may make the controller an attractive attack target, it also allows

defensive policies to be applied dynamically across the network [1], [3], [5]. The layered architecture used as the theoretical basis of the project is illustrated in Figure 2.1.

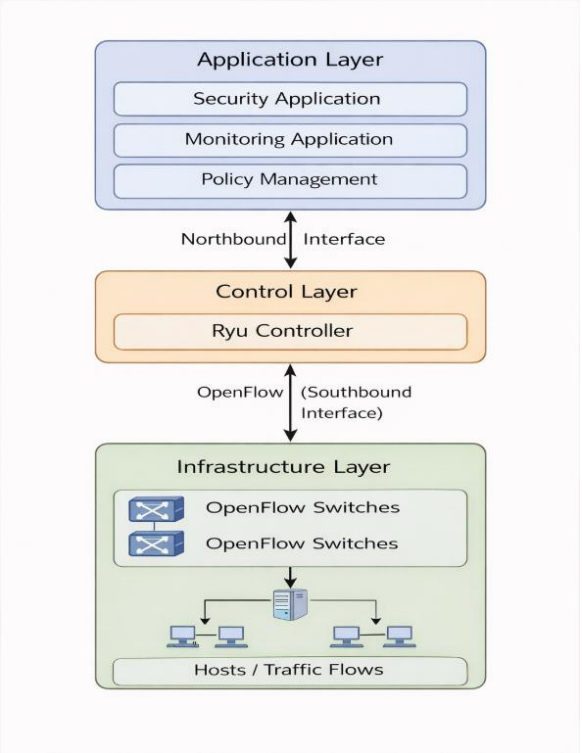


Figure 2.1 Basic SDN architecture used as the theoretical basis of the project, adapted from[1].

2.2 OpenFlow-Based Monitoring and Enforcement

OpenFlow connects the SDN controller with programmable switches and enables the controller to install, modify, and remove flow rules dynamically. In the proposed framework, it translates controller decisions into actions applied within the Open vSwitch data plane [1], [3].

Open vSwitch traffic mirroring sends a copy of selected traffic to Snort without changing the original forwarding path. Snort inspects the mirrored traffic, while Ryu remains responsible for network control and mitigation.

The framework applies graduated enforcement through normal forwarding, three rate-limiting levels implemented with OpenFlow meters, and high-priority drop rules for severe malicious

traffic. Thus, OpenFlow supports both monitoring integration and adaptive defense enforcement.

2.3 Intrusion Detection and SDN Security Integration

The Ryu controller represents the central decision component of the implemented SDN framework. It maintains network-flow information, receives security evidence, evaluates the observed traffic state, and applies the selected mitigation action through OpenFlow. Because centralized controllers may be affected by excessive Packet-In requests and flooding traffic, controller-aware detection and mitigation are essential in SDN security [1], [3], [5].

The original project phase focused on DDoS, ICMP flooding, and IP-spoofing behavior. Snort inspected mirrored traffic and generated alerts when predefined suspicious patterns were detected. A forwarding agent then transferred normalized alert information from the Snort virtual machine to the Ryu controller, separating traffic inspection from controller-side decision making [1].

This design forms the detection and control foundation retained in the extended project. Snort alerts and controller-observed traffic indicators continue to support the original security workflow, while the current Artificial Intelligence component provides more detailed identification of multi-stage attack families and behavioral subpatterns. The theoretical basis of this AI component is presented in the subsequent sections

2.4 Game Theory and Attacker-Defender Interaction

Game Theory provides a mathematical framework for analyzing strategic interactions between entities whose decisions affect one another. In cybersecurity, the attacker attempts to increase disruption or avoid detection, while the defender seeks to reduce damage without unnecessarily affecting legitimate traffic. Therefore, selecting a defensive response involves balancing protection, service continuity, and mitigation cost rather than applying the strongest action to every suspicious event [1], [4], [7].

In the implemented framework, the attacker is represented by a host or group of hosts generating traffic that may evolve through normal, probing, flooding, spoofing, or combined

malicious behavior. The defender is represented by the Ryu controller supported by the dynamic game engine. Based on the evaluated network state, the defender selects one of the available strategies: ALLOW, RL_1, RL_2, RL_3, or BLOCK.

The current game engine extends the original attacker–defender model by considering repeated observations and changes in attack behavior. Its decision process uses network indicators, Snort evidence, spoofing information, attack intensity, source activity, and the detected attack stage. Markov-based transition information is also used to represent how the observed behavior may move from one stage to another. The detailed theoretical basis of these stage transitions is discussed in Section 2.6.

This adaptive formulation is consistent with research that combines Game Theory with the Cyber Kill Chain to analyze attacker and defender strategies across different stages of a cyberattack [9]. In the proposed project, Game Theory therefore forms the decision layer between attack detection and OpenFlow enforcement.
To clarify the role of Game Theory in the extended SDN security framework, Table 2.1 maps the main attacker–defender elements, observed attack states, and available defense strategies to their corresponding components in the implemented project.

Table 2.1 Game-Theoretic Mapping Used in the Proposed Project.

Game Element	**Project Mapping**
Attacker	A host or group of hosts generating suspicious or malicious network traffic
Defender	The Ryu controller supported by the dynamic game engine
Observed attack states	NORMAL, PROBE, FLOOD_LOW, FLOOD_HIGH, IP_SPOOFING, and SPOOFED_FLOOD
Defender Strategies	ALLOW, RL_1, RL_2, RL_3, and BLOCK.
Main Observations	Packet rate, Snort alert, spoofing information, packet loss, RTT, active sources, current stage, and previous stage
Defender Objective	Reduce attack impact while preserving legitimate traffic and avoiding unnecessary mitigation
Attacker Objective	Increase disruption, conceal malicious behavior, or avoid effective mitigation
Decision Logic	Evaluate the observed state, transition behavior, and strategy payoffs before selecting the defense action

2.5 Multi-Stage Cyberattacks

multi-stage cyberattack consists of a sequence of related behaviors that develop over time rather than appearing as one isolated malicious event. Different stages may use different attack vectors, tools, traffic characteristics, or levels of intensity. Consequently, representing the complete attack using one general label may hide the detailed behaviors that compose the attack scenario and make detection more difficult [8].

The Cyber Kill Chain provides a stage-oriented view of cyberattacks by describing how attacker behavior progresses through a sequence of activities. When combined with Game Theory, these stages can also be used to analyze how attacker and defender strategies change during the attack process [9]. In this project, the same general principle is used to distinguish changes in observed network behavior and support an adaptive response rather than treating all suspicious traffic as one fixed attack state.

The implemented framework represents the observed behavior through the states NORMAL, PROBE, FLOOD_LOW, FLOOD_HIGH, IP_SPOOFING, and SPOOFED_FLOOD. These states describe the traffic condition observed by the controller; they are not presented as the formal stages of the Cyber Kill Chain. The controller maintains a history for each monitored destination and compares the previous state with the current state to identify meaningful transitions, including changes between flooding and spoofing-related behavior.

Multi-stage evidence is then used together with traffic indicators, Snort alerts, active-source information, and Artificial Intelligence output. This allows the framework to distinguish an isolated observation from an evolving attack sequence and provides additional context for the subsequent Markov and Game Theory decision processes.

The relationship between the progression of multi-stage attack behavior and the corresponding security-state tracking process is illustrated in Figure 2.2.

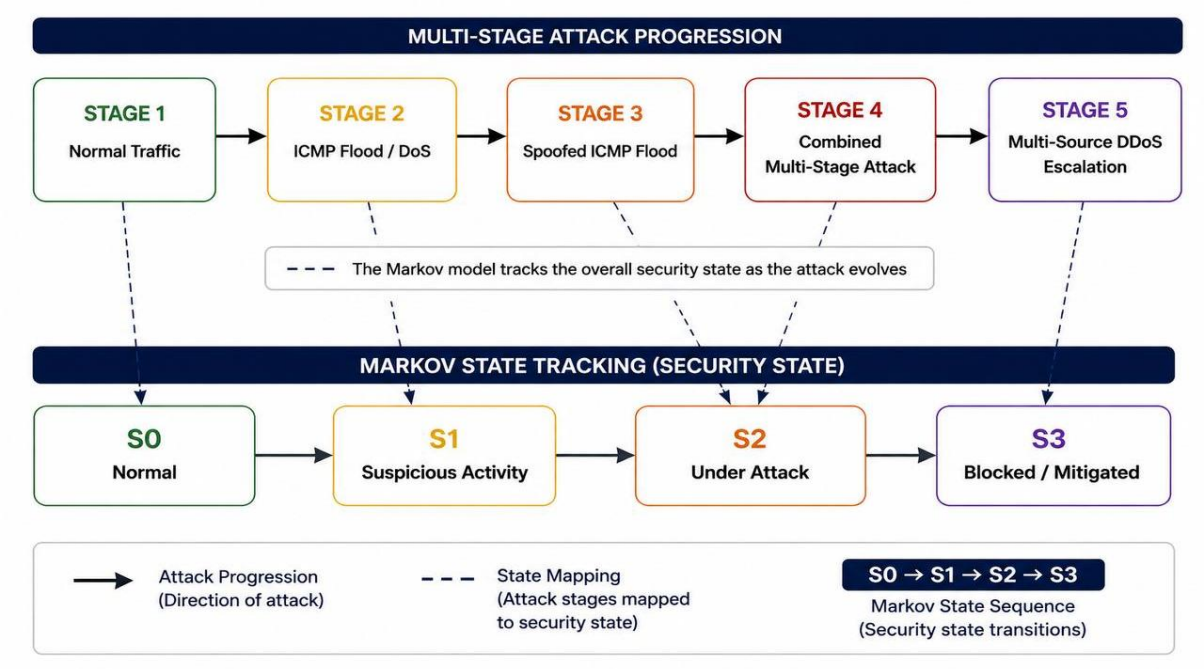


Figure 2.2 Illustrative relationship between multi-stage attack progression and Markov-based security-state tracking in the project.

2.6 Markov-Based Attack Stage Modeling

A Markov model represents a dynamic system through a set of states and the probabilities of transition between them. Its basic assumption is that the probability of the next state depends on the current state. This makes it suitable for analyzing network behavior that changes across successive observation rounds rather than evaluating each traffic event independently.

The transition from a current state Si to a subsequent state Sj can be represented as:

$$P(X_{t+1} = S_j \mid X_t = S_i)$$

where Xt denotes the security state at time t, and Pij denotes the probability of moving from state Si to state Sj. Depending on the observed evidence, the system may remain in its current state, move toward a more severe state, or return to a less severe state.

As illustrated previously in Figure 2.2, the progression of multi-stage attack behavior is mapped to general security states that represent normal traffic, suspicious activity, a detected attack, and blocked or mitigated traffic. This simplified representation explains the relationship between attack progression and temporal state tracking. It does not imply that

every attack must pass through all states in one fixed sequence.

In the operational implementation, the dynamic game engine uses the more detailed traffic states NORMAL, PROBE, FLOOD_LOW, FLOOD_HIGH, IP_SPOOFING, and SPOOFED_FLOOD. These states describe the detected behavior and its severity more precisely than the general conceptual states shown in Figure 2.2. The transition matrix assigns probabilities to movement between the operational states, including the probability of remaining in the same state.

During each decision round, the framework evaluates the previous stage together with the current traffic evidence, including traffic intensity, spoofing indicators, Snort alerts, active-source information, and available Artificial Intelligence output. The Markov result provides temporal context about the evolving attack condition to the Game Theory engine. Markov modeling therefore supports stage tracking and state estimation, while Game Theory remains responsible for selecting the appropriate defense action.

2.7 Artificial Intelligence for Network Intrusion Detection

Artificial Intelligence-based intrusion detection uses data-driven models to learn the statistical characteristics of normal and malicious network traffic. Unlike signature-based detection, which depends on predefined rules, machine learning models can classify traffic according to patterns extracted from the available features. This makes AI useful for analyzing large network-flow datasets and identifying complex behaviors that may not be represented by one fixed signature.

However, detecting multi-stage attacks remains difficult because different attack behaviors may have similar statistical distributions, and some malicious patterns may resemble normal traffic. In addition, assigning one broad label to all traffic associated with an attack campaign may hide the detailed behaviors that form the attack. These limitations can reduce the ability of a classifier to separate normal traffic from complex attack patterns accurately [8].

The AI component developed in this project addresses this problem through a multi-stage processing pipeline. The selected network-traffic data is cleaned and transformed, informative features are prepared, and Principal Component Analysis is applied before attack

pattern segmentation. The resulting subpatterns are then used for ensemble-based and hierarchical classification. The final detection output includes the predicted attack family, the identified subpattern, and the associated prediction confidence.

The AI component does not replace the existing Snort and Ryu-based security framework. Instead, it provides a more detailed data-driven analysis that complements the original detection indicators. Its output is subsequently transferred to the existing defense process, where Markov-based stage information and Game Theory are used to support the selection of an appropriate mitigation action.

2.8 Network-Traffic Feature Engineering

Network-traffic feature engineering transforms raw or flow-based observations into numerical characteristics that are more suitable for machine learning. This step is important because the original flow representation may lose temporal and behavioral information when a complex attack campaign is summarized into independent traffic records [8].

Port information is particularly useful in multi-stage attack analysis because changes in source and destination port usage can reflect the services, tools, and activities involved in different attack behaviors. However, individual port numbers are highly localized and may not be directly suitable as machine-learning inputs. Therefore, statistical properties of port behavior can provide a more general representation than the raw values themselves [8].

The reference study calculates port variance over sliding windows to capture changes in network behavior at different observation scales. Smaller windows describe short-term fluctuations, while larger windows represent broader behavioral characteristics. The resulting variance values are added to the traffic-flow dataset as engineered features to support the separation of normal and multi-stage attack patterns [8].
The effect of port-variance feature extraction over a larger sliding window is illustrated in Figure 2.3.

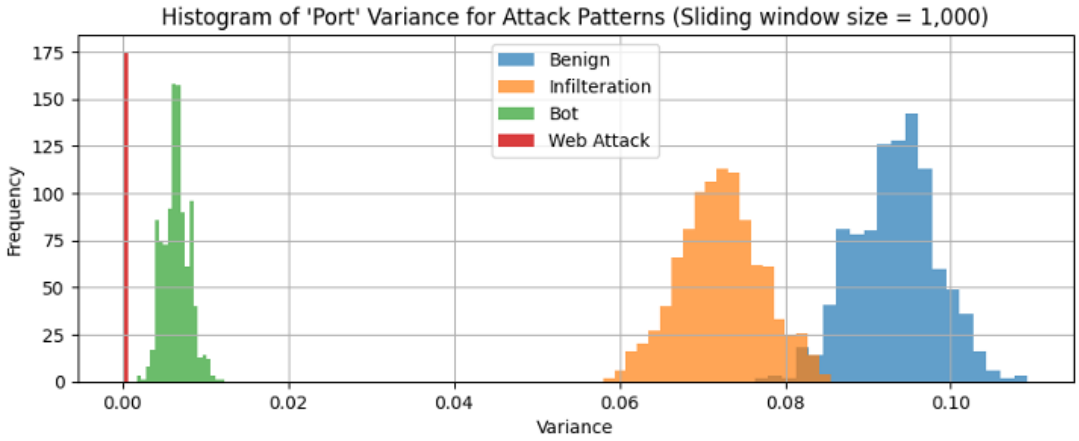


Figure 2.3. Port-Variance Distributions for Benign and Multi-Stage Attack Patterns Using a Sliding Window of Size 1,000, Adapted from [8]

In the implemented project pipeline, port-variance features are generated after data cleaning and before dimensionality reduction. This preserves additional behavioral information that may help distinguish Benign, Bot, Web Attacks, and Infiltration traffic before the data is standardized and transformed using Principal Component Analysis.

2.9 Principal Component Analysis

Network-traffic datasets commonly contain a large number of numerical features, some of which may be correlated or provide redundant information. Processing all available features can increase computational cost and may make the subsequent clustering and classification stages more difficult. Principal Component Analysis (PCA) addresses this issue by transforming the original features into a smaller set of uncorrelated variables called principal components.

Each principal component represents a linear combination of the original features. The first component preserves the largest possible amount of variance in the data, while each subsequent component preserves the greatest remaining variance under the condition that it is uncorrelated with the preceding components. Therefore, PCA reduces dimensionality while retaining the main statistical structure of the dataset.

The reference study applies PCA before clustering because packet- and flow-based datasets may contain redundant statistical information. Dimensionality reduction improves the

efficiency of the attack pattern segmentation process and provides a compact representation for distinguishing detailed attack behaviors [8].

In the implemented project, the engineered numerical features are first standardized using StandardScaler. Incremental PCA is then fitted in batches so that the large compressed dataset can be processed without loading all records into memory simultaneously. A maximum of 30 principal components is considered during fitting, and the cumulative explained variance is evaluated to determine how many components preserve the required information.

The first 26 principal components are used in the subsequent attack pattern segmentation and classification stages because they retain at least 95% of the cumulative explained variance. The transformed dataset therefore contains PC1 through PC26 together with the corresponding traffic label, providing a reduced and standardized representation for the following stages.

2.10 Attack Pattern Segmentation

Attack pattern segmentation is the main contribution of the reference study and represents the foundation of the proposed Artificial Intelligence pipeline. Instead of treating every attack family as a single homogeneous class, this approach divides each attack family into several behavioral subpatterns that share similar statistical characteristics. As a result, the classifier learns simpler and more consistent patterns rather than attempting to model highly diverse attack behaviors within one class [8].

Conventional machine learning classifiers often struggle with multi-stage attacks because different attack procedures may produce substantially different traffic distributions while still belonging to the same attack family. Consequently, assigning a single label to all traffic associated with an attack family may increase intra-class variation and reduce classification accuracy. Attack pattern segmentation addresses this limitation by grouping samples with similar behavioral characteristics before the classification stage [8].

The segmentation process is performed independently for each attack family after dimensionality reduction. Samples that exhibit similar statistical behavior are grouped

together using clustering techniques, producing a set of representative subpatterns for each family. These subpatterns preserve the detailed behavioral structure of the original attack while reducing the variability observed within the complete attack family.
The overall segmentation-based detection methodology proposed in the reference study is illustrated in Figure 2.4.

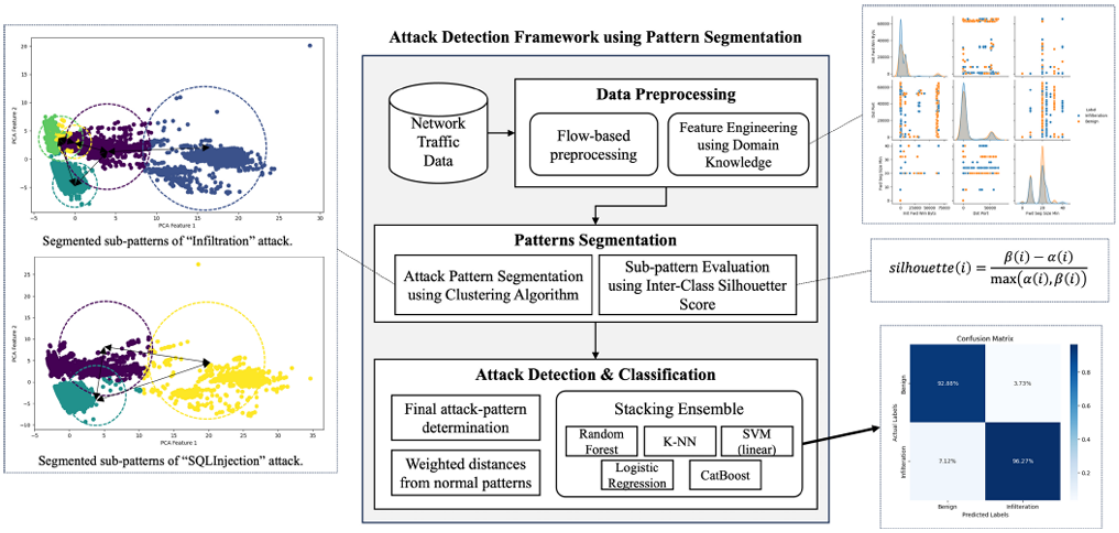


Figure 2.4. Overview of the Attack Pattern Segmentation-Based Network Attack Detection Technique, Adapted from [8]

In the implemented project, attack pattern segmentation is applied after PCA using class-specific clustering models. Separate clustering is performed for Benign, Bot, Web Attacks, and Infiltration, allowing each attack family to be divided into its own behavioral subpatterns. Each network-flow record is therefore assigned both an attack family label and a corresponding subpattern label, which are subsequently used during hierarchical classification.

2.11 Ensemble and Stacking Classification

Ensemble learning combines the predictions of multiple machine learning models to obtain a more reliable final decision. The main purpose is to benefit from the complementary strengths of different classifiers rather than depending on one model whose performance may vary across different traffic distributions.

Stacking is an ensemble method organized into two learning levels. At the first level, several

base classifiers are trained on the same input features. Each classifier learns the data from a different perspective and produces prediction probabilities or intermediate predictions. At the second level, these outputs are provided to a meta-learner, which learns how to combine them and produce the final classification result.

The stacking framework presented in the reference study uses Random Forest, k-Nearest Neighbors, Logistic Regression, Linear Support Vector Machine, and CatBoost as base learners. These models were selected to capture linear, nonlinear, tree-based, and local-neighborhood relationships within the segmented attack data. A meta-learner then combines their outputs to determine the final subpattern and its corresponding parent attack pattern [8]. The two-level stacking classification process adopted as the initial AI detection mechanism is illustrated in Figure 2.5.

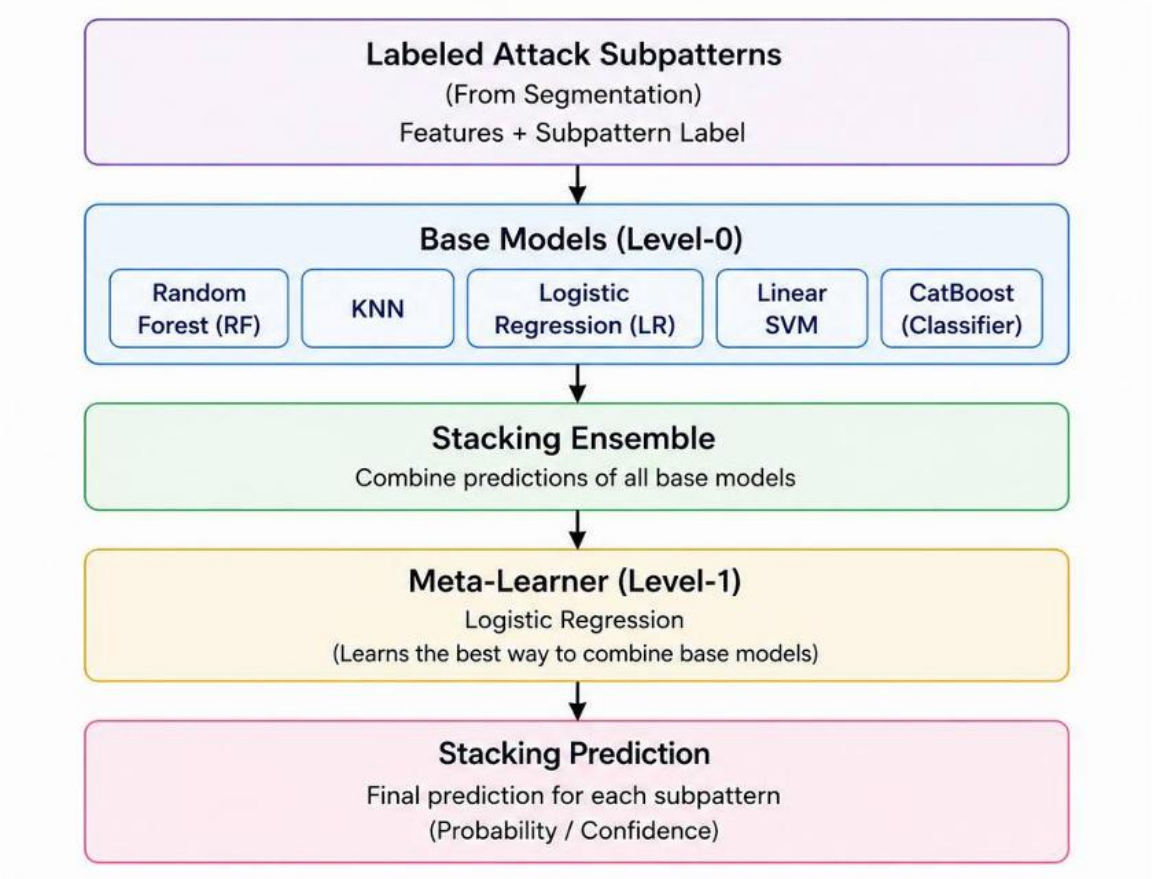


Figure 2.5. Two-Level Stacking Ensemble Used for Segmented Attack-Pattern Classification

In the project, stacking represents the initial Artificial Intelligence classification stage applied to the segmented dataset. The input consists of the principal-component features and the

generated subpattern labels. The resulting model is evaluated to determine how effectively the segmented attack behaviors can be distinguished before developing the improved hierarchical classification mechanism presented in the following section.

2.12 Hierarchical Attack Classification

Hierarchical classification organizes the prediction task into multiple related levels rather than attempting to identify every detailed class through one flat classifier. This structure is suitable when the target classes have a natural parent–child relationship. In this project, each behavioral subpattern belongs to one of four parent traffic families: Benign, Bot, Web Attacks, or Infiltration.

The improved hierarchical classifier operates through two prediction levels. At Level 1, a CatBoost model receives the selected principal-component features and predicts the general traffic family. At Level 2, the predicted family determines which family-specific CatBoost model is used to identify the detailed subpattern. Therefore, the second-level model distinguishes only between the subpatterns belonging to the selected family rather than comparing all subpatterns simultaneously.

This organization reduces the complexity of the detailed prediction task and preserves the relationship between each subpattern and its parent family. It also allows the training configuration and class-balancing strategy to be adjusted according to the characteristics of each family. Separate second-level models are therefore trained for Benign, Bot, Web Attacks, and Infiltration traffic.

The final hierarchical output contains both the predicted attack family and the corresponding behavioral subpattern. The probability produced at each level is retained so that the system can evaluate the reliability of the complete hierarchical prediction.

2.13 Prediction Confidence and Joint Scoring

Prediction confidence indicates how strongly a classification model supports its selected class. In the hierarchical classifier, confidence is produced at two related levels. The Level-1 model provides the probability of the predicted attack family, while the selected Level-2

model provides the probability of the predicted subpattern within that family.

Considering only one of these probabilities may provide an incomplete measure of the final prediction. A high subpattern probability is not sufficient when the parent-family prediction is uncertain, and a high family probability does not guarantee that the detailed subpattern has been identified reliably. Therefore, the two probabilities are combined into a joint score:

Joint Score = Family Probability × Subpattern Probability

The joint score represents the confidence of the complete family–subpattern prediction path. A high value indicates that both the parent family and the detailed subpattern are supported by their corresponding models. Conversely, a lower value may result from uncertainty at either classification level.

In the implemented classifier, a family-confidence threshold of 0.90 is used. When the highest family probability is equal to or greater than this threshold, the corresponding family-specific model is used to generate the final subpattern prediction. When the family confidence is below 0.90, the two most probable family candidates are considered. Their corresponding subpattern predictions and joint scores are calculated, and the family–subpattern combination with the higher joint score is selected.

The final output therefore includes the predicted attack family, the predicted subpattern, and the joint score as the overall confidence value. Low-confidence predictions are also recorded during evaluation because they may indicate overlapping traffic distributions, weak subpatterns, or uncertainty between closely related attack behaviors.

2.14 Integration of AI Detection with SDN Defense

The Artificial Intelligence component extends the original SDN security framework by providing a more detailed description of the detected traffic. Instead of producing only a general malicious or benign decision, the hierarchical classifier identifies the predicted attack family, the corresponding behavioral subpattern, and the joint confidence score.

The detection result is prepared by the integration bridge and transmitted through an HTTP POST request to the receiver deployed in the VMware environment. The receiver validates the incoming information and forwards the accepted detection result to the dynamic Game Theory engine. This communication mechanism allows the AI analysis and the SDN defense components to operate as separate modules while exchanging the information required for adaptive decision-making.

The end-to-end integration between AI-based detection and SDN-based defense is illustrated in Figure 2.6.

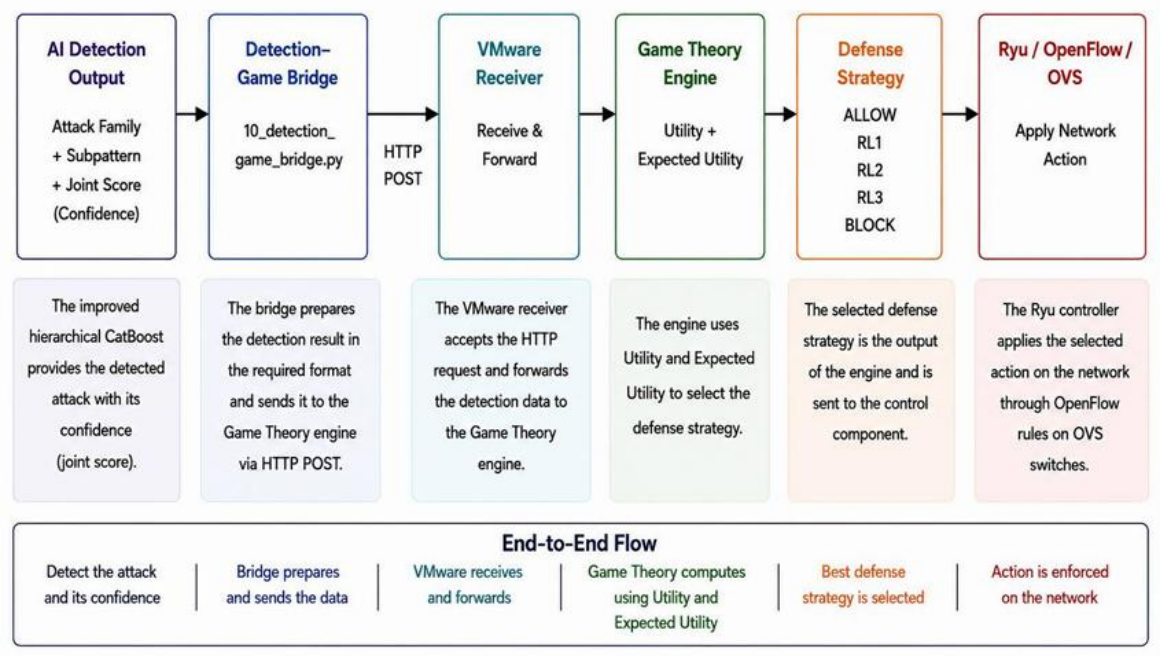


Figure 2.6. End-to-End Integration of AI Detection, Game Theory, and SDN-Based Defense

The Game Theory engine combines the AI result with the available network observations, including traffic intensity, Snort alerts, packet loss, RTT, spoofing indicators, active-source information, the current attack state, and the previous state. The AI result is accepted only when an attack is detected and the joint confidence satisfies the configured acceptance condition. Therefore, the machine-learning output is treated as an additional source of evidence rather than the only basis for the defense decision.

After evaluating the possible defense strategies, the engine selects one of the available actions: ALLOW, RL1, RL2, RL3, or BLOCK. The selected action is then delivered to the

Ryu controller, which translates the decision into OpenFlow rules and applies it through the Open vSwitch infrastructure. This completes the end-to-end process from traffic analysis and attack identification to adaptive network enforcement.

2.15 Chapter Summary

This chapter presented the theoretical foundation of the project, including SDN, OpenFlow, intrusion detection, Game Theory, and Markov-based state tracking. It also introduced the Artificial Intelligence concepts used in the extended phase, including feature engineering, PCA, attack pattern segmentation, stacking, hierarchical classification, and confidence scoring.

The chapter further explained how the AI detection output is integrated with the existing SDN defense framework to support adaptive response through Game Theory, Ryu, OpenFlow, and Open vSwitch. The next chapter presents the implementation environment, dataset, system components, and methodology.

3 Chapter Three: Working Environment

3.1 Introduction

This chapter presents the working environment and implementation methodology of the complete security framework. The original SDN environment, including Mininet, Open vSwitch, Ryu, Snort, Game Theory, and OpenFlow-based mitigation, is summarized as the foundation of the implemented system.

The framework was first extended with multi-stage attack tracking using operational security states, previous and current stage information, and Markov-based transition logic to represent the evolution of suspicious behavior across successive decision rounds.

Artificial Intelligence was subsequently introduced as a second extension. The AI pipeline performs data preparation, feature engineering, PCA, attack pattern segmentation, ensemble classification, and hierarchical CatBoost-based prediction to identify the attack family, behavioral subpattern, and associated confidence.

Finally, the AI detection output is integrated with the existing multi-stage, Game Theory, and SDN defense mechanisms to support adaptive decision making and programmable network enforcement.

3.2 Overall Project Implementation Environment

The complete project was implemented as two connected environments. The first environment represents the original Software-Defined Networking security platform and includes the Mininet network emulator, Open vSwitch, the Ryu controller, Snort IDS, the alert-forwarding mechanism, and the dynamic Game Theory engine. This environment is responsible for network emulation, traffic monitoring, threat evaluation, defense selection, and OpenFlow-based enforcement.

The second environment contains the Artificial Intelligence processing and classification pipeline. It is responsible for preparing the CIC-IDS-2018 traffic data, generating engineered features, applying dimensionality reduction, segmenting attack patterns, training the classification models, and producing the final attack-family, subpattern, and confidence output.

The complete implementation environment and the connection between the AI processing pipeline and the SDN defense platform are illustrated in Figure 3.1.

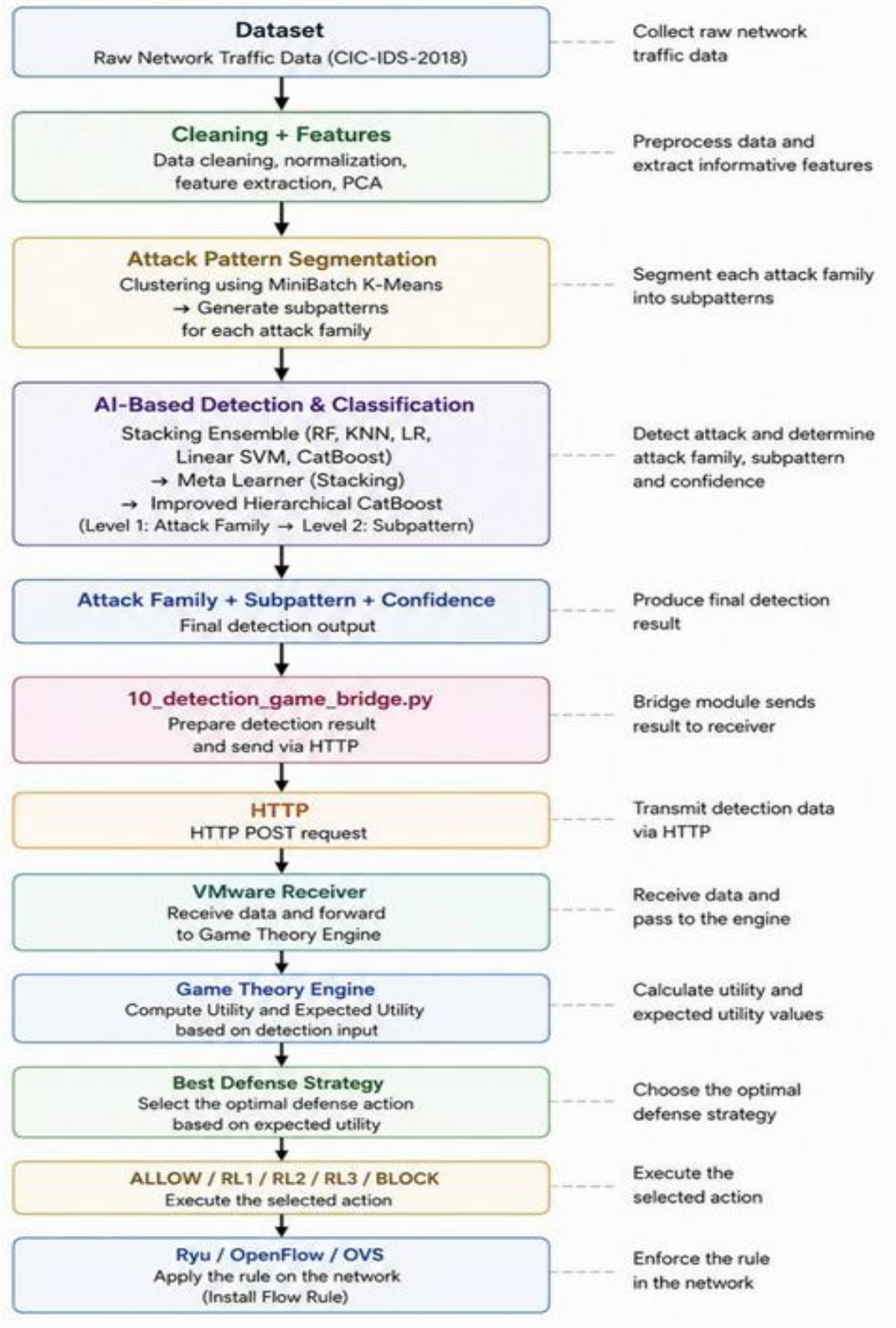


Figure 3.1. Complete AI-Driven Multi-Stage Attack Detection and SDN Defense Environment

The two environments are connected through an integration mechanism that transfers

accepted AI detection results to the dynamic defense system. The Game Theory engine combines this information with the observations collected from the SDN environment, after which the selected defense action is enforced by the Ryu controller through OpenFlow rules and Open vSwitch.

3.3 Software and Development Components

The project was developed using a combination of virtualization, Software-Defined Networking, intrusion-detection, data-processing, and machine-learning technologies. The original environment provides network emulation, traffic inspection, controller-side decision-making, and OpenFlow enforcement, while the added Artificial Intelligence environment performs dataset preparation, feature transformation, attack pattern segmentation, model training, and prediction generation.

Python is the main development language shared by the two environments. It is used for the Ryu controller applications, alert-forwarding and integration scripts, Game Theory logic, data-processing stages, and machine-learning models. Table 3.1 summarizes the main software components and their roles in the complete project.

Table 3.1 ***Software and Development Components Used in the Complete Project***

Role in the Project	Component / Technology	Environment
Hosts the virtual machines used for the SDN, monitoring, and integration environments	VMware Workstation	Virtualization
Provides the Linux environment used by the Mininet, Snort, and Ryu components	Ubuntu 22.04.5 LTS	Operating System
Implements the controller, agents, Game Theory logic, data processing, integration, and machine-learning scripts	Python 3.10.12	Development Language
Creates the emulated network topology containing hosts and OpenFlow switches	Mininet 2.3.0	SDN Environment
Performs packet forwarding, traffic	Open vSwitch 2.17.9	Data Plane

Role in the Project	Component / Technology	Environment
mirroring, meter-based rate limiting, and flow-rule enforcement		
Connects the Ryu controller to Open vSwitch and supports flow and meter operations	OpenFlow 1.3	Southbound Protocol
Collects network observations and applies the selected mitigation actions	Ryu 4.34	SDN Controller
Inspects mirrored traffic and generates alerts for suspicious behavior	Snort 3.1.5.0	Intrusion Detection
Generates ICMP flooding traffic during practical attack experiments	hping3 3.0.0-alpha-2	Attack Generation
Generates customized and spoofed packets for security-testing scenarios	Scapy 2.4.4	Packet Crafting
Loads, cleans, filters, transforms, and stores the network-traffic datasets	pandas	Data Processing
Supports numerical conversion, array operations, and invalid-value handling	NumPy	Numerical Processing
Provides data splitting, standardization, PCA, clustering, evaluation metrics, and supporting classifiers	scikit-learn	Machine Learning
Implements the family-level and family-specific hierarchical classification models	CatBoost	Gradient Boosting
Saves and loads scalers, PCA objects, clustering models, and trained classifiers	joblib	Model Storage
Transfers accepted AI detection results to the dynamic defense environment	HTTP-based Python bridge and receiver	AI–Defense Integration

Role in the Project	Component / Technology	Environment
Combines network and AI evidence and selects ALLOW, RL_1, RL_2, RL_3, or BLOCK	Dynamic Game Theory engine	Decision Module

The SDN-related versions in Table 3.1 were retained from the original implementation environment. The Artificial Intelligence libraries were added according to their actual functions in the project source code. Their installed package versions should be recorded directly from the final execution environment to preserve reproducibility and avoid reporting versions that were not used during model development.

The selected components support a modular implementation. Network emulation and live defense operate within the virtualized SDN environment, whereas computationally intensive dataset preparation and model training are executed through the AI pipeline. The saved preprocessing and model objects allow later stages to reuse the same transformations and classification logic without repeating the complete training process.

3.4 Original SDN Experimental Environment

The original project phase was implemented in a virtualized Software-Defined Networking environment. The environment combined Mininet, Open vSwitch, the Ryu controller, Snort IDS, an alert-forwarding agent, and a dynamic Game Theory engine. Its purpose was to monitor network traffic, detect suspicious behavior, evaluate the current threat condition, and apply an adaptive mitigation action.

Mininet was used to emulate the network hosts and switches, while Open vSwitch provided OpenFlow-based forwarding and traffic mirroring. Mirrored traffic was delivered to Snort for inspection, and generated alerts were forwarded to the controller environment. The Ryu controller then combined IDS evidence with network observations and transferred the resulting state information to the Game Theory decision engine.

The available defense actions were ALLOW, RL_1, RL_2, RL_3, and BLOCK. The selected action was converted into OpenFlow flow rules or meter rules and applied to the

corresponding switch. This original environment forms the defense platform that receives and uses the Artificial Intelligence detection output developed in the current project phase.

The main components of the original SDN security environment are illustrated in Figure 3.2.

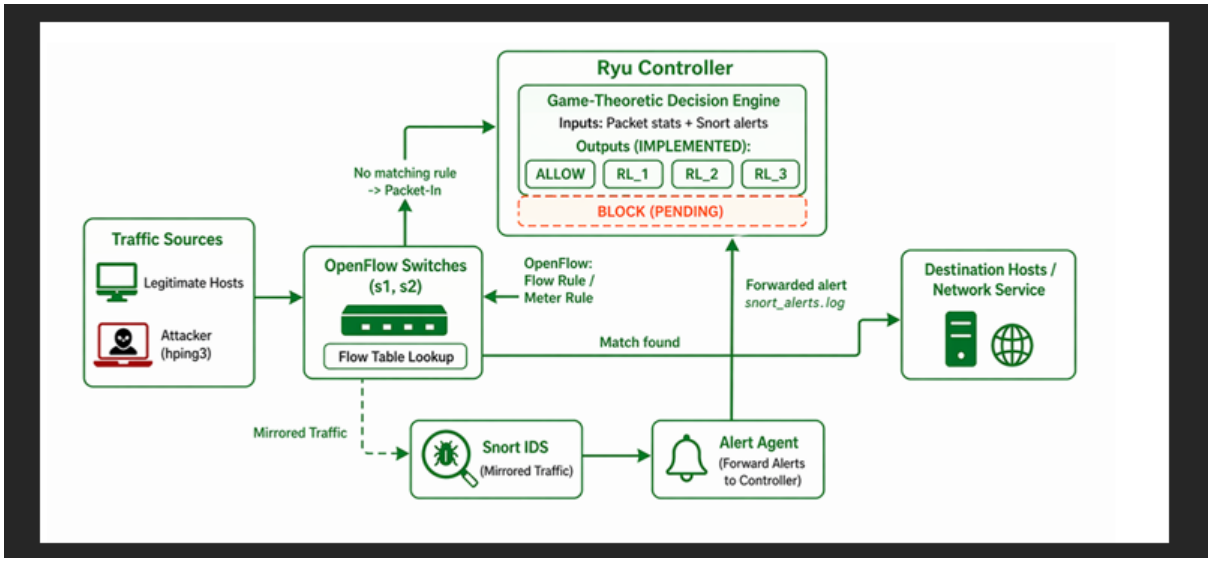


Figure 3.2. Original SDN Monitoring, Detection, Decision, and Enforcement Environment

3.5 Virtual Machines and Network Interface Layout

The original SDN environment was distributed across separate virtual machines to isolate network emulation, traffic inspection, and controller-side processing. The Mininet virtual machine hosted the emulated topology, Open vSwitch instances, and the Ryu controller, while the Snort virtual machine was dedicated to inspecting mirrored network traffic.

Each virtual machine used a management interface for normal communication and additional interfaces for monitoring and traffic mirroring. On the Mininet virtual machine, separate interfaces were assigned to the mirrored outputs of the two OpenFlow switches. On the Snort virtual machine, dedicated monitoring interfaces received the copied traffic generated by the mirroring configuration.

This arrangement separated management communication from monitored traffic and allowed Snort to inspect traffic from both switches without interfering with the normal forwarding path.

The configured interfaces of the Mininet and Snort virtual machines are shown in Figures 3.3 and 3.4.

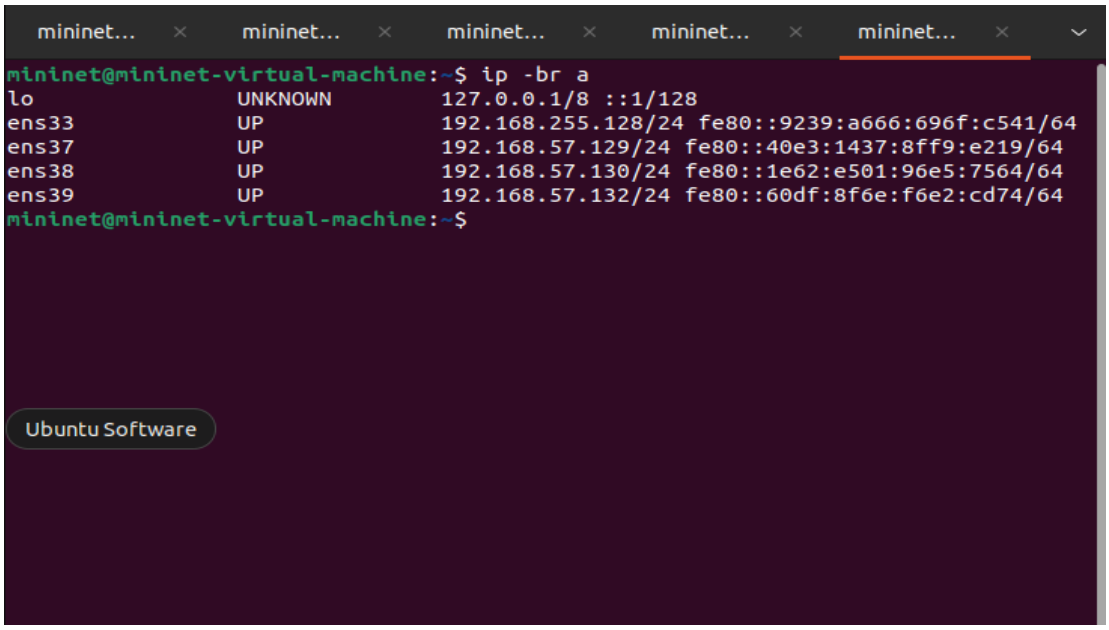


Figure 3.3. Network Interfaces Configured on the Mininet Virtual Machine

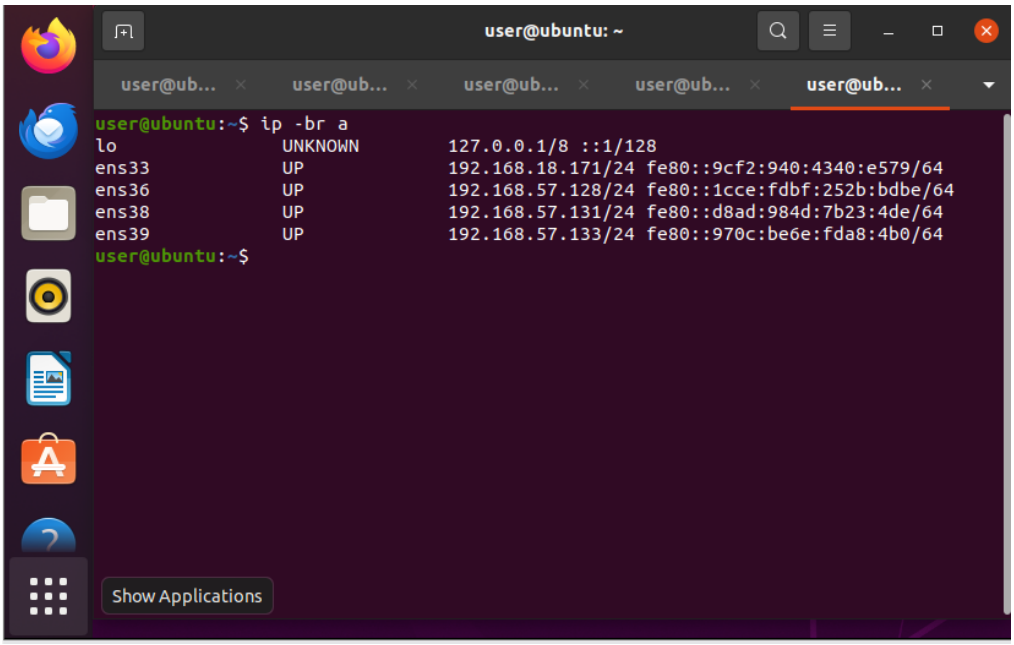


Figure 3.4. Management and Monitoring Interfaces Configured on the Snort Virtual Machine

Table 3.2. Virtual-Machine Interfaces Used for Management and Traffic Monitoring

VM	Interface	Purpose
Mininet VM	ens33	Management / Internet access
Mininet VM	ens37	Mirror output port for s1
Mininet VM	ens39	Mirror output port for s2
Snort VM	ens33	Management / SSH access
Snort VM	ens36	Monitoring interface for mirrored traffic
Snort VM	ens38	Monitoring interface for mirrored traffic

The dedicated monitoring interfaces support traffic visibility from both OpenFlow switches, while the management interfaces remain available for controller communication, remote access, and system administration. This arrangement separates management communication from monitored traffic and allows Snort to inspect traffic from both switches without interfering with the normal forwarding path.

3.6 SDN Topology and Security Components

The original SDN topology was implemented using two OpenFlow switches connected to a centralized Ryu controller. Six virtual hosts were distributed across the two switches, where hosts h1, h2, and h3 were connected to switch s1, while hosts h4, h5, and h6 were connected to switch s2. During the attack experiments, h1 was used as the main attacker host and h4 as the primary victim.

The topology also included the main security components required for monitoring and mitigation. Open vSwitch mirrored traffic from both switches to the Snort virtual machine, while the alert-forwarding agent transferred the generated IDS alerts to the Ryu controller. The dynamic Game Theory engine then evaluated the network condition and selected an appropriate defensive action.

The topology and the interaction between the monitoring, detection, decision, and enforcement components are illustrated in Figure 3.5.

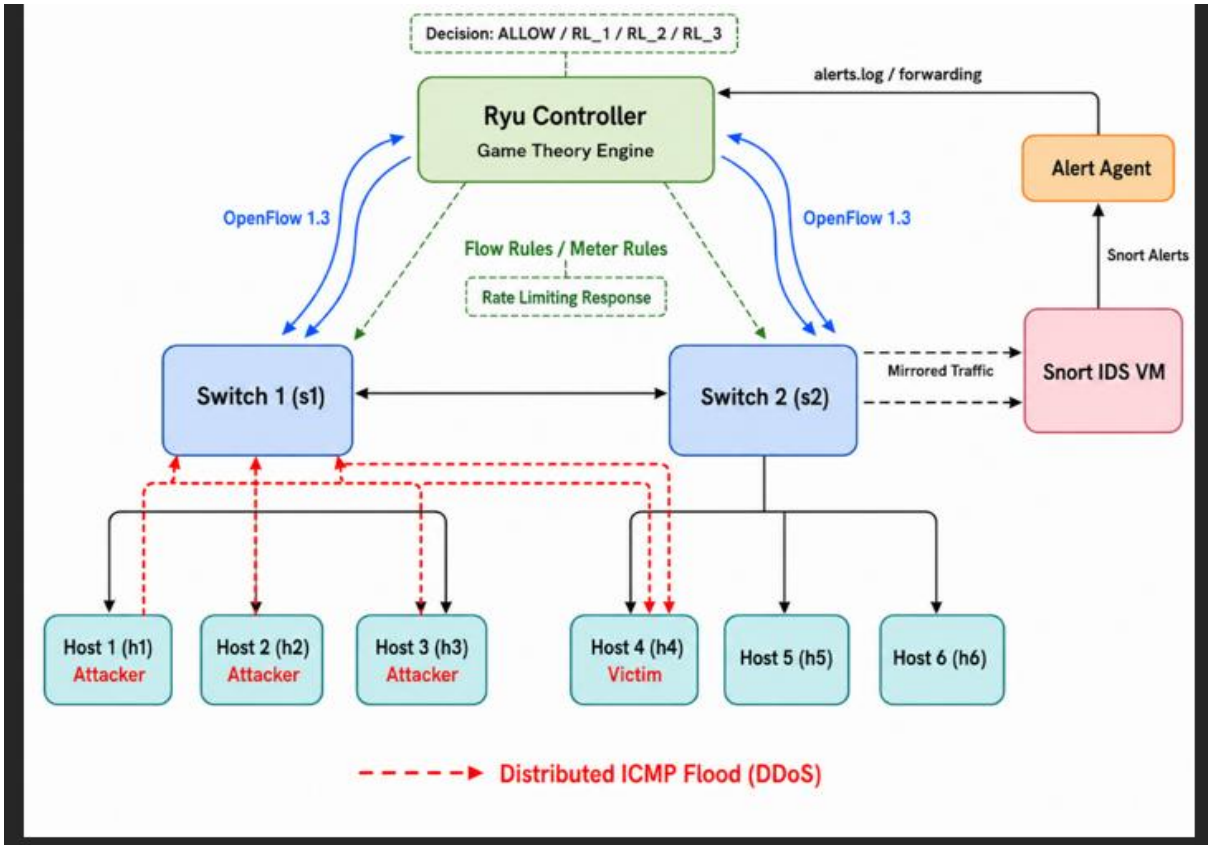


Figure 3.5. SDN Topology and Security-Component Integration

The available defense actions were ALLOW, RL_1, RL_2, RL_3, and BLOCK. Rate-limiting decisions were enforced through OpenFlow meter rules, while severe or persistent malicious traffic could be blocked using a high-priority drop rule. This topology therefore provided the practical network environment required to evaluate adaptive attack detection and mitigation.

3.7 Artificial Intelligence Development Environment

The Artificial Intelligence extension was developed as an independent processing pipeline that complements the original SDN security platform. Unlike the SDN environment, which operates on live network observations and controller-side decisions, the AI environment focuses on dataset preparation, feature transformation, model training, evaluation, and prediction generation.

The implementation was developed using Python together with Pandas, NumPy, Scikit-learn, CatBoost, and Joblib. These tools were used for reading compressed datasets, cleaning and transforming features, applying Principal Component Analysis, segmenting attack patterns,

training classification models, evaluating predictions, and saving the trained processing objects.

The AI environment was organized as a sequence of executable Python modules. Each module performs a specific stage of the workflow and produces an output file that becomes the input of the following stage. This modular structure reduces code dependency, simplifies testing, and allows individual processing stages to be repeated without executing the complete pipeline again.
The modular organization of the Artificial Intelligence implementation and the generated processing outputs are shown in Figure 3.6.

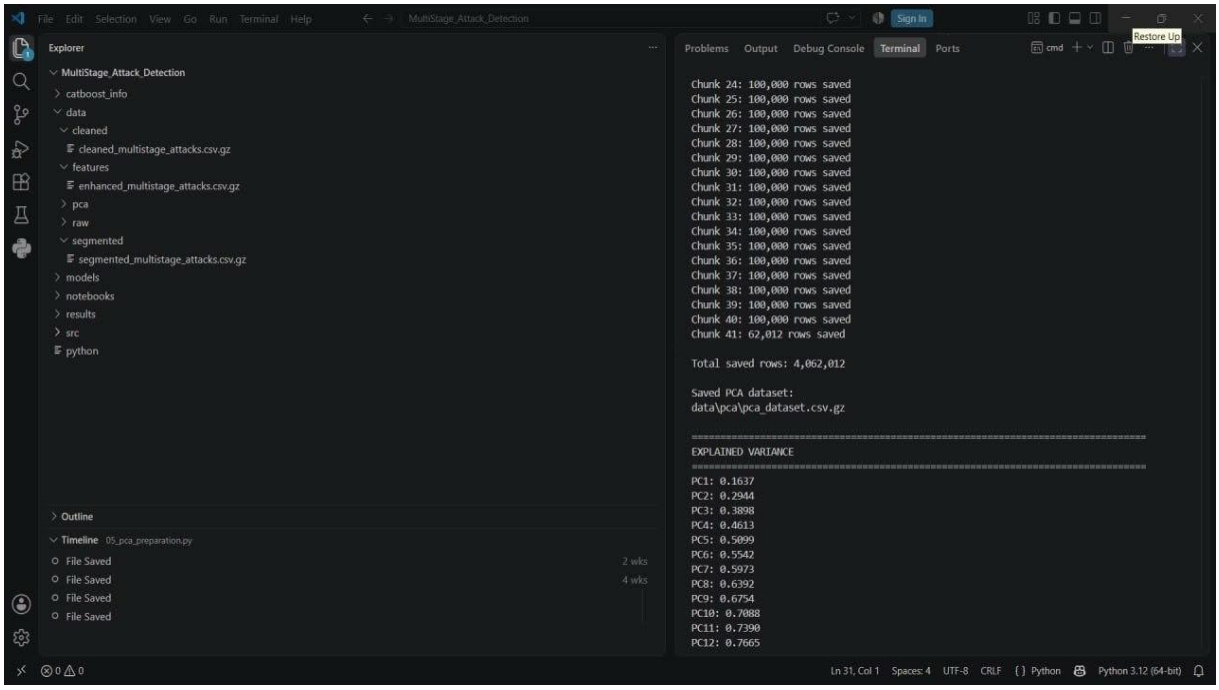


Figure 3.6. Modular Organization and Execution Output of the Artificial Intelligence Project

The main generated datasets are stored in compressed CSV format to reduce storage requirements and support efficient processing. The models, preprocessing objects, evaluation reports, and intermediate outputs are stored in separate project directories to maintain a clear and reproducible implementation structure.

3.8 Dataset Selection and Target Attack Families

The Artificial Intelligence pipeline was developed using network-traffic records derived from the CIC-IDS-2018 dataset. This dataset was selected because it contains benign traffic

together with several realistic attack categories and provides a suitable basis for studying complex and multi-stage attack behavior.

The current project focuses on four traffic families: Benign, Bot, Infiltration, and Web Attacks. These families represent different behavioral characteristics and were also included in the adopted attack-pattern segmentation methodology. Benign represents legitimate network activity, whereas Bot, Infiltration, and Web Attacks represent malicious traffic with different levels of complexity and similarity to normal behavior.

Table 3.3. Traffic Families Selected for the Artificial Intelligence Pipeline

Role in the Project	Type	Traffic Family
Represents legitimate network behavior	Normal	Benign
Represents automated bot-related attack behavior	Malicious	Bot
Represents malicious activity that may resemble normal traffic	Malicious	Infiltration
Represents attacks targeting web applications and services	Malicious	Web Attacks

The class-selection module processed the original CIC-IDS-2018 traffic files in chunks and retained only the records belonging to the four target families. Processing the data in chunks reduced memory consumption and allowed the large dataset files to be handled efficiently. The selected records were then combined and stored in a compressed dataset for the subsequent cleaning and feature-engineering stages.

The execution output of the target-class selection stage and the final number of retained records are shown in Figure 3.7.

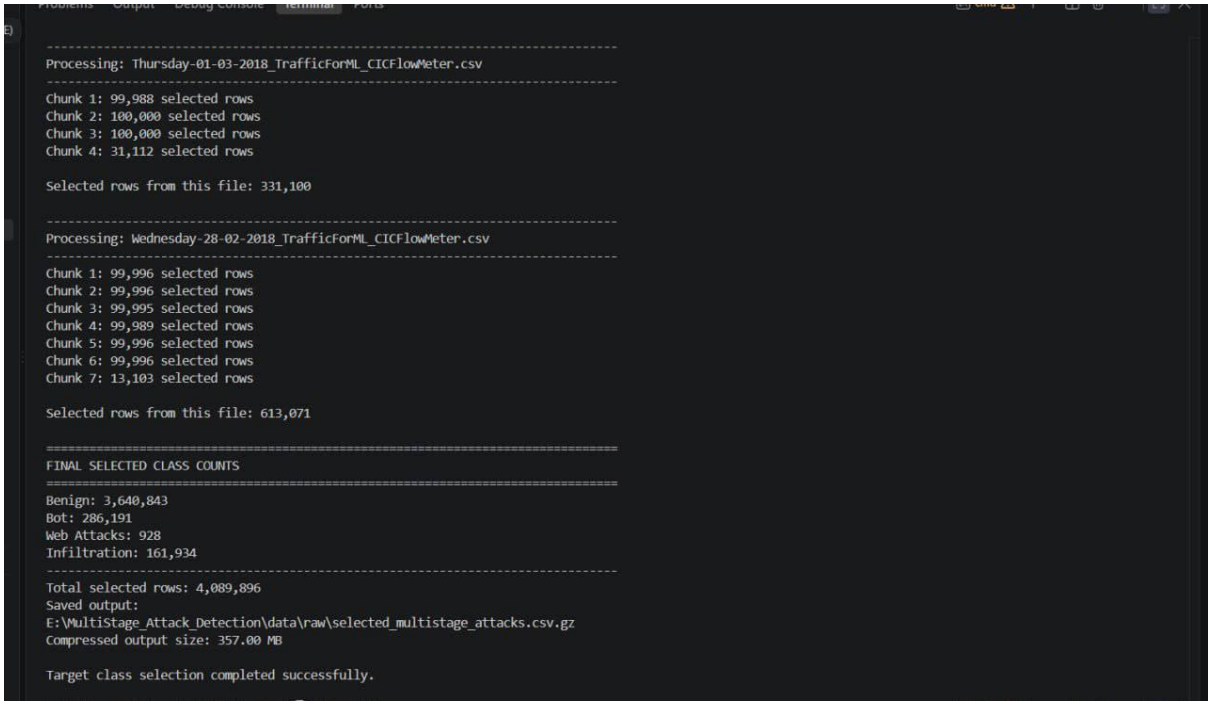


Figure 3.7. Target Traffic-Family Selection and Dataset Generation Output

The target-class selection stage retained a total of 4,089,896 traffic records. The selected data consisted of 3,640,843 Benign records, 286,191 Bot records, 161,934 Infiltration records, and 928 Web Attacks records. The resulting dataset was saved as selected_multistage_attacks.csv.gz and used as the input to the data-cleaning stage.

After the selection stage, the retained records passed through cleaning, feature engineering, dimensionality reduction, and attack-pattern segmentation. The final segmented dataset was stored as segmented_multistage_attacks.csv.gz and contained the principal-component features, the original traffic-family label, the assigned cluster identifier, and the generated subpattern label. This dataset was later used as the direct input to the stacking and hierarchical classification models.

3.9 AI Data-Processing Pipeline

The Artificial Intelligence implementation follows a sequential data-processing pipeline in which the output of each stage becomes the input of the next stage. The pipeline begins with the selection of the target traffic families from the CIC-IDS-2018 dataset and continues through data cleaning, feature engineering, standardization, dimensionality reduction, attack-pattern segmentation, model training, and final prediction generation.

The pipeline was designed as a modular sequence of Python scripts. Each script performs one clearly defined operation and stores its output in a dedicated project directory. This organization allows individual stages to be tested, repeated, or modified without rerunning the complete workflow from the beginning.

As previously illustrated in Figure 3.1, the Artificial Intelligence workflow is organized as a sequential pipeline extending from dataset preparation to prediction generation and integration with the SDN defense environment.

The first stage selects the four target traffic families and combines the retained records into a compressed dataset. The cleaning stage then standardizes column names and labels, removes invalid or missing values, and prepares the numeric traffic features for further processing. A port-variance feature is subsequently generated to provide additional information about communication behavior before the data is standardized and transformed using Principal Component Analysis.

The reduced feature representation is then divided into behavioral subpatterns using class-specific clustering. The segmented dataset is used to train the stacking ensemble and the hierarchical CatBoost models. During prediction, the system first identifies the traffic family and then predicts the corresponding subpattern within that family. The resulting prediction includes the family label, subpattern label, and confidence values required by the subsequent integration stage.

The following sections describe each stage of the Artificial Intelligence pipeline in greater detail, beginning with data cleaning and label preparation.

3.10 Data Cleaning and Label Preparation

The selected CIC-IDS-2018 records were cleaned before feature engineering and model development. This stage was necessary because the original traffic files could contain inconsistent column names, label variations, missing values, infinite values, and non-numeric entries that could negatively affect the subsequent processing stages.

The cleaning module first standardized the dataset structure by removing unnecessary spaces from column names and traffic labels. The retained labels were then normalized so that records belonging to the same traffic family used a consistent class name. In particular, the required web-attack variants were consolidated under the Web Attacks family.

The traffic-feature columns were converted into numeric values. Entries that could not be converted were treated as missing values, while positive and negative infinity values were

replaced with undefined values. Rows containing invalid values in the required feature or label columns were then removed to ensure that only valid records continued to the feature-engineering stage.

The execution output and final summary of the data-cleaning stage are shown in Figure 3.8.

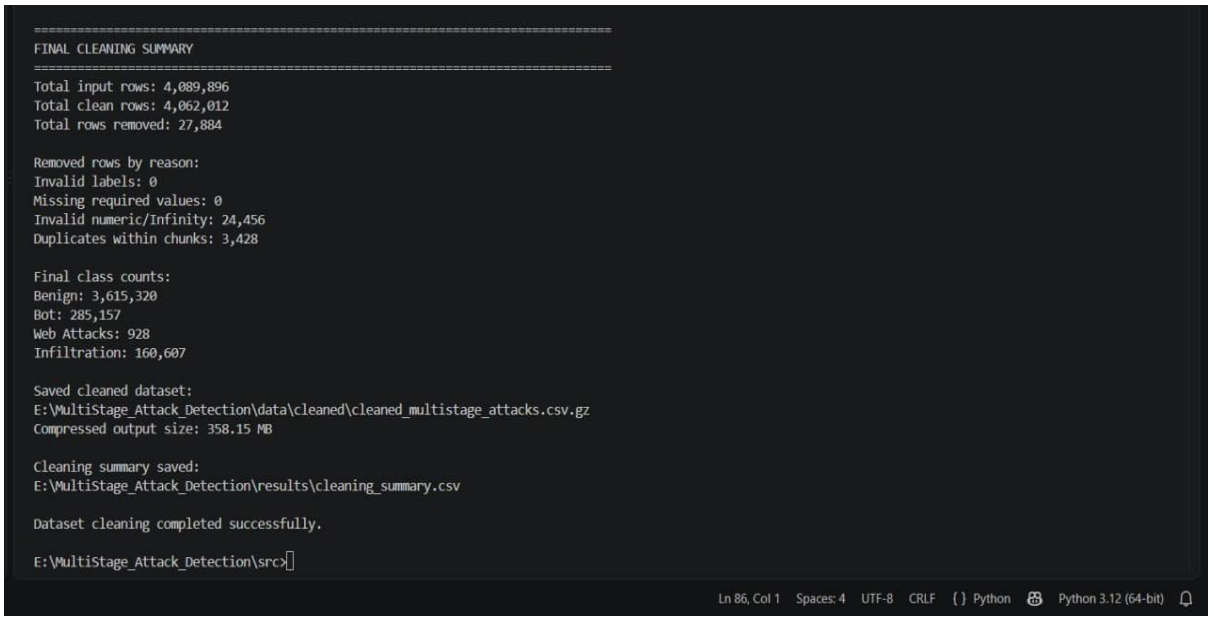


Figure 3.8. Final Data-Cleaning Summary and Cleaned-Dataset Generation Output

The cleaning stage processed 4,089,896 input records and retained 4,062,012 valid records. A total of 27,884 records were removed, including 24,456 records containing invalid numeric or infinite values and 3,428 duplicate records. No records were removed because of invalid labels or missing required values. The cleaned dataset was saved as cleaned_multistage_attacks.csv.gz, while a separate cleaning summary was stored in cleaning_summary.csv.

The cleaned dataset became the input to the port-variance feature-engineering stage.

3.11 Port-Variance Feature Engineering

After the data-cleaning stage, an additional traffic feature was generated to improve the representation of communication behavior. The selected feature was based on the variance of port information, which can help describe changes in service usage and communication patterns across different types of network traffic.

Port information is relevant because different applications, services, and attack tools may produce distinct communication patterns. Instead of using individual port numbers directly, the implementation generated a statistical representation of port behavior. This approach reduces dependence on specific port values while preserving information about changes in network activity.

The feature-engineering module processed the cleaned dataset in chunks and calculated the required port-variance values before appending them to the existing traffic features. Chunk-based processing was used to reduce memory consumption and support the large number of retained traffic records.
The execution output and final summary of the port-variance feature-engineering stage are shown in Figure 3.9.

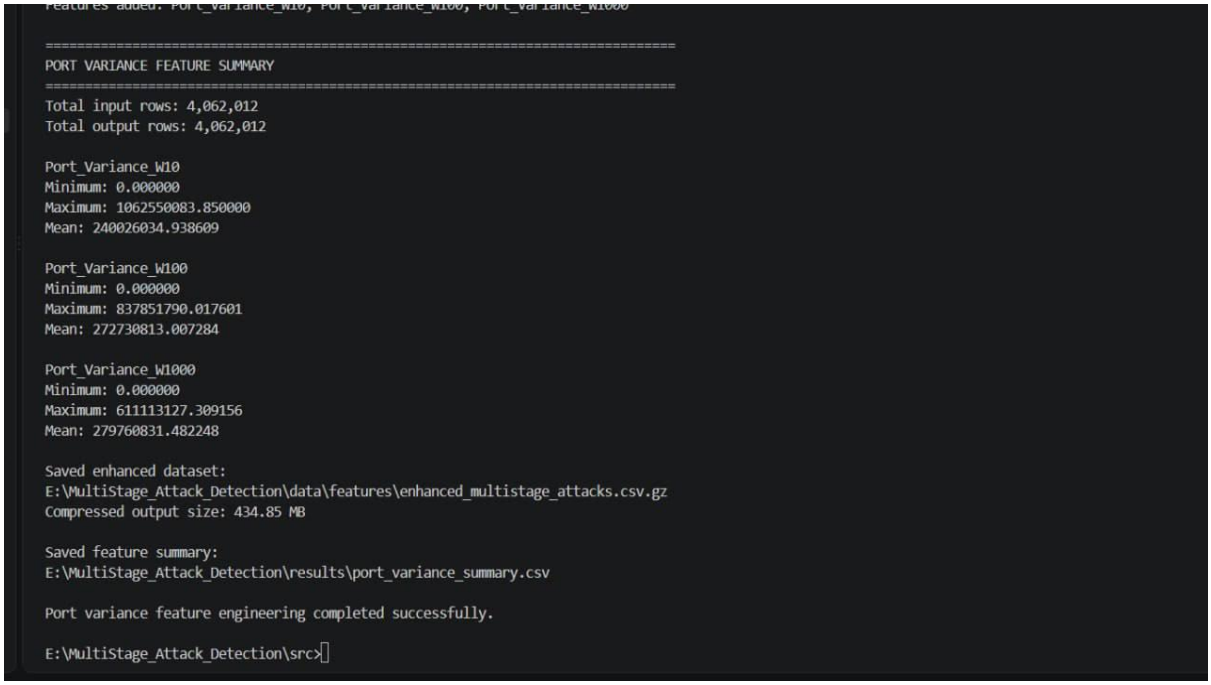


Figure 3.9. Port-Variance Feature Engineering Summary and Enhanced-Dataset Generation Output

The feature-engineering stage processed all 4,062,012 cleaned records and generated three additional features: Port_Variance_W10, Port_Variance_W100, and Port_Variance_W1000. The enhanced dataset was saved as enhanced_multistage_attacks.csv.gz, while the statistical summary of the generated features was stored in port_variance_summary.csv.

The enhanced dataset became the input to the standardization and Principal Component Analysis stage.

3.12 Data Standardization and PCA Preparation

Before applying attack-pattern segmentation and classification, the enhanced traffic features were standardized and transformed using Principal Component Analysis. Standardization was required because the original features had different numerical ranges, which could cause features with large values to dominate the subsequent learning and clustering processes.
The StandardScaler was fitted incrementally on the enhanced dataset. The feature values were processed in chunks rather than loading the complete dataset into memory. Each valid chunk contributed to the estimation of the global feature means and standard deviations, after which the same fitted scaler was used to transform the records.
Incremental Principal Component Analysis was then applied to the standardized features. The implementation initially allowed a maximum of 30 principal components and selected the minimum number of components required to preserve at least 95% of the cumulative explained variance. This reduced the dimensionality of the traffic data while retaining most of its meaningful information.

Table 3.4. Standardization and PCA Configuration

Configured Value	Parameter
enhanced_multistage_attacks.csv.gz	Input dataset
Chunk-based incremental processing	Processing method
100,000 records	Chunk size
Standard Scaler	Standardization method
Incremental PCA	PCA method
30	Maximum tested components
95%	Explained-variance threshold
PC1–PC26	Retained components
pca_dataset.csv.gz	Output dataset

The resulting representation retained 26 principal components, identified as PC1 through PC26. The fitted StandardScaler and Incremental PCA objects were saved so that the same transformations could be reused during later prediction and integration stages.

The final dimensionality-reduction summary and the generated PCA dataset are shown in Figure 3.10.

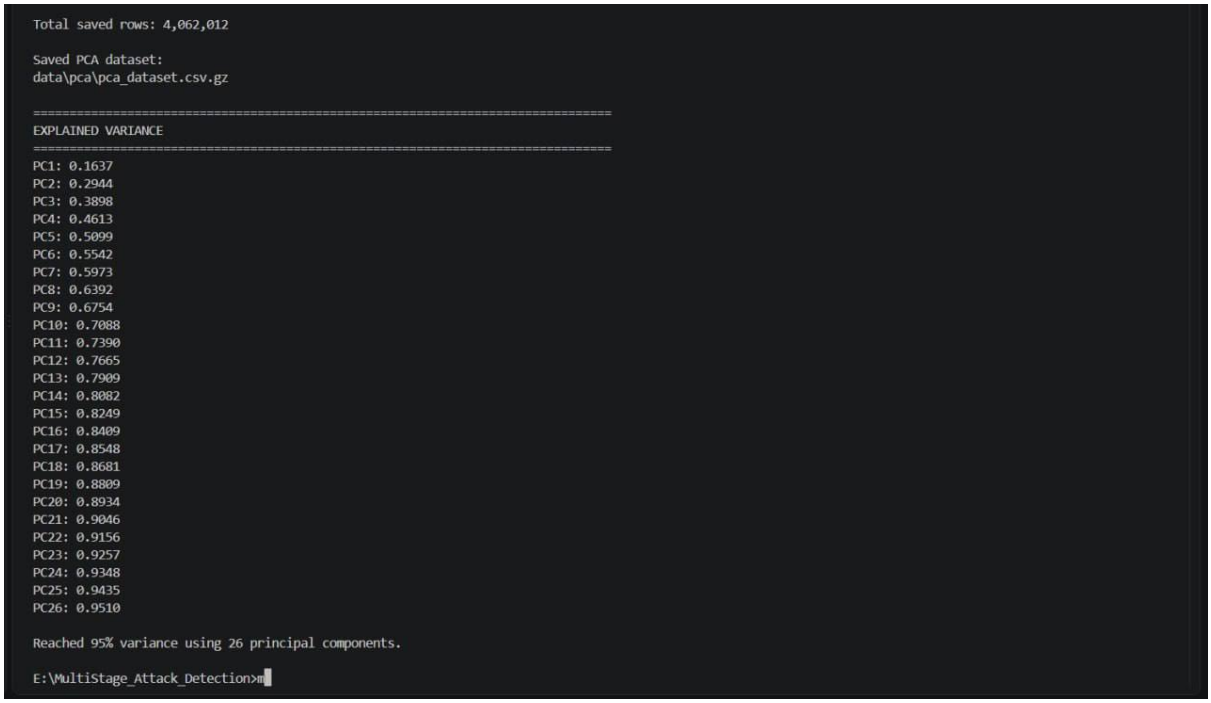


Figure 3.10. Explained-Variance Summary and PCA Dataset Generation Output

The PCA stage processed and saved 4,062,012 traffic records. The cumulative explained variance reached 95.10% using 26 principal components. The transformed dataset was saved as pca_dataset.csv.gz and became the input to the attack-pattern segmentation stage.

3.13 Attack Pattern Segmentation

After dimensionality reduction, the PCA-transformed traffic records were divided into behavioral subpatterns. The purpose of this stage was to represent the internal behavioral diversity of each traffic family rather than treating all records belonging to the same family as a single homogeneous class.

Attack-pattern segmentation was performed independently for the four traffic families: Benign, Bot, Infiltration, and Web Attacks. A separate clustering model was therefore used for each family so that its records could be grouped according to their own behavioral characteristics in the PCA feature space.

K-Means-based clustering was applied to the principal-component features, and each resulting cluster was assigned a descriptive subpattern label. The generated labels followed the format Family_Pattern_n, allowing every record to retain both its original traffic-family identity and its more specific behavioral pattern.

Table 3.5. Generated Behavioral Subpatterns

Traffic Family	Number of Subpatterns	Generated Subpatterns	Total Records
Benign	2	Benign_Pattern_1 – Benign_Pattern_2	3,615,320
Bot	5	Bot_Pattern_1 – Bot_Pattern_5	285,157
Infiltration	9	Infiltration_Pattern_1 Infiltration_Pattern_9	160,607
Web Attacks	6	Web Attacks_Pattern_1 – Web Attacks_Pattern_6	928
Total	22	_	4,062,012

The segmentation stage assigned a subpattern label and a cluster identifier to every processed traffic record. The resulting dataset preserved the 26 principal-component features together with the original traffic-family label, the assigned Cluster_ID, and the generated Subpattern label.

The final subpattern distribution and the generated segmented dataset are shown in Figure 3.11.

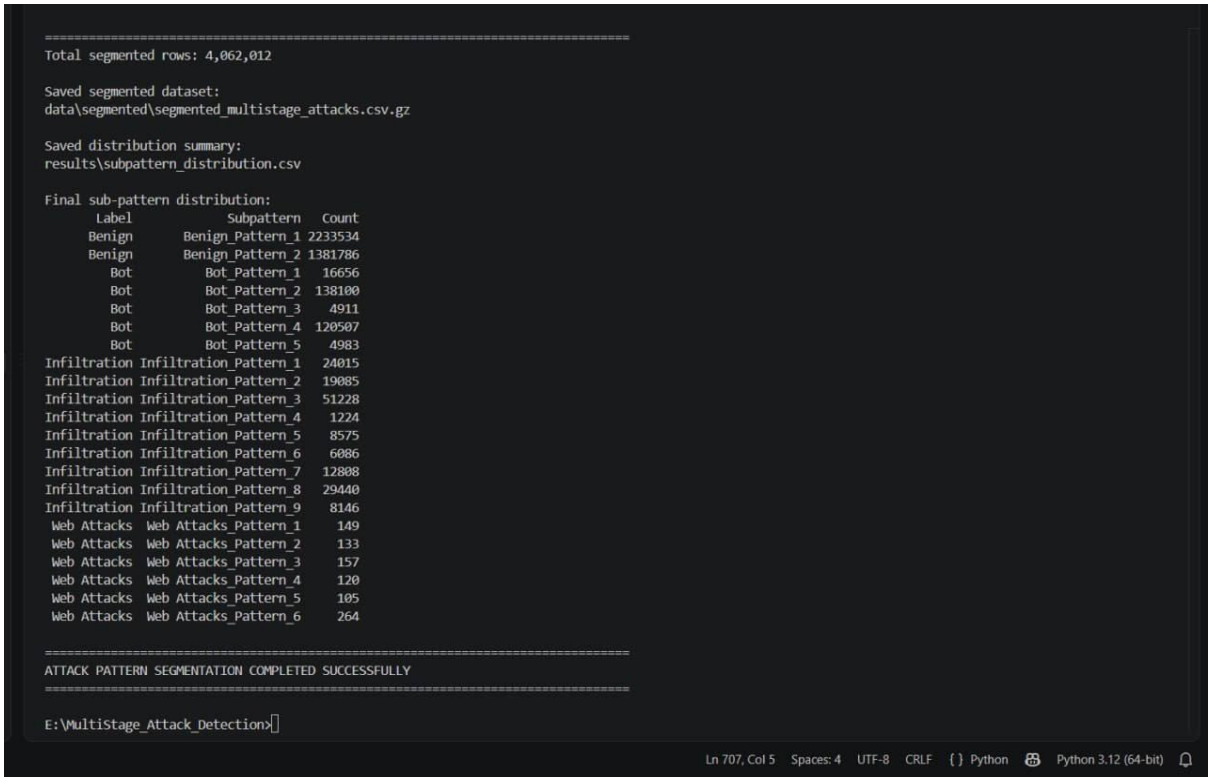


Figure 3.11. Final Attack-Pattern Segmentation and Subpattern Distribution

The segmentation stage successfully processed all 4,062,012 records and generated a total of 22 behavioral subpatterns. Benign traffic was divided into 2 subpatterns, Bot traffic into 5 subpatterns, Infiltration traffic into 9 subpatterns, and Web Attacks into 6 subpatterns. The final segmented dataset was saved as segmented_multistage_attacks.csv.gz, while the detailed subpattern distribution was stored in subpattern_distribution.csv.

The segmented dataset became the direct input to the machine-learning classification stage, where the generated subpatterns were used as the prediction targets for the stacking and hierarchical classification models.

3.14 Stacking Ensemble Implementation

Following attack-pattern segmentation, a stacking ensemble architecture was adopted to classify the generated behavioral subpatterns. Instead of relying on a single machine-learning algorithm, the proposed framework combines several complementary classifiers to improve the robustness and generalization capability of the detection process.

The input to the stacking model consisted of the 26 principal components together with the generated subpattern labels obtained during the segmentation stage. These transformed

features provide a compact representation of the original network traffic while preserving the most significant behavioral information required for classification.
The stacking architecture combines Random Forest, k-Nearest Neighbors, Logistic Regression, Linear Support Vector Machine, and CatBoost. Each classifier represents a different learning strategy. Random Forest and CatBoost capture complex nonlinear relationships, k-Nearest Neighbors identifies local similarities between behavioral patterns, while Logistic Regression and Linear SVM provide linear and margin-based classification perspectives.

Each base classifier independently generated predictions for the segmented traffic records. The outputs of these classifiers were then combined by a stacking meta-learner to produce the final subpattern prediction. This approach allows the framework to exploit the complementary strengths of multiple classification techniques instead of depending on the behavior of a single model.
The trained stacking architecture was incorporated into the overall Artificial Intelligence detection framework and provided an additional classification mechanism for identifying detailed behavioral subpatterns.

3.15 Hierarchical CatBoost Implementation

A hierarchical CatBoost classification architecture was implemented as an additional detection model for the segmented traffic data. The purpose of this architecture was to divide the prediction process into two consecutive levels: traffic-family classification followed by detailed subpattern classification.

The model used the segmented dataset generated in the previous stage. The principal components PC1 through PC26 were used as the input features, while the generated Subpattern field was used to derive both the parent traffic family and the detailed behavioral class.

At the first level, a CatBoost classifier was trained to distinguish among the four traffic families: Benign, Bot, Infiltration, and Web Attacks. After the family was identified, the second level used a dedicated CatBoost classifier associated with that family to predict the corresponding behavioral subpattern. Therefore, the final output contains both a general

traffic-family prediction and a more specific subpattern prediction.

To reduce computational requirements while preserving the distribution of the available classes, a stratified sample of up to 400,000 records was used during model development. A fixed random state of 42 was applied for reproducibility, while subpatterns containing fewer than 10 records were excluded to reduce instability during training.

A family-confidence threshold of 0.90 was incorporated into the hierarchical prediction process. This threshold allows the framework to identify cases in which the first-level family prediction is uncertain and to handle these predictions more carefully during the subsequent classification stage.

After training, the family classifier, family-specific subpattern classifiers, feature definitions, and confidence configuration were stored as a reusable model object for later prediction and integration with the Game Theory decision module.

3.16 Prediction Output and Confidence Generation

After model training, the detection framework generated structured prediction outputs for the evaluated traffic records. The prediction process produced both a parent traffic-family label and a detailed behavioral subpattern label, enabling the system to describe detected activity at two different levels.

In addition to the predicted labels, confidence values were generated to represent the reliability of each classification decision. The family-level confidence represents the certainty associated with the first-stage prediction, while the subpattern-level confidence represents the certainty of the detailed prediction generated within the selected traffic family.

The framework also generates a joint-confidence indicator that summarizes the overall reliability of the hierarchical prediction. This information is important because the subsequent defense mechanism requires more than a simple malicious-or-benign decision. A prediction associated with high confidence can be treated differently from a prediction whose classification remains uncertain.

The final prediction information therefore contains the detected traffic family, the corresponding behavioral subpattern, and the associated confidence indicators. These values form the structured output that is transferred to the subsequent integration stage.

This design allows the Artificial Intelligence subsystem to provide detailed and confidence-aware detection information rather than producing only a binary attack decision.

3.17 AI-to-Game-Theory Integration

The Artificial Intelligence subsystem was integrated with the Game Theory decision module to enable adaptive security responses based on the detected traffic behavior. Instead of using the classification output only for reporting purposes, the prediction result was transformed into structured decision-support information that could be used by the defense layer.

The integration stage transfers the predicted traffic family, the detected behavioral subpattern, and the corresponding confidence indicators. It also communicates whether an attack was detected, allowing the Game Theory module to evaluate both the type of suspicious activity and the reliability of the Artificial Intelligence decision.

The exchanged information is organized in a structured format to maintain a consistent interface between the machine-learning environment and the SDN-based security platform. This structure simplifies the transfer of prediction information between independently implemented components of the system.

The integration stage therefore acts as a bridge between the Artificial Intelligence detection subsystem and the adaptive defense mechanism. Once a prediction is generated, the relevant information is prepared and forwarded to the Game Theory component for further evaluation and response selection.

This architecture separates detection from decision making. The Artificial Intelligence subsystem identifies and characterizes the observed network behavior, while the Game Theory subsystem uses the received information to determine an appropriate defensive action.

3.18 Dynamic Game Theory Decision Module

Following the Artificial Intelligence prediction stage, the generated detection information was forwarded to the Dynamic Game Theory module. This component is responsible for selecting an appropriate defensive strategy according to the detected attack behavior and the confidence associated with the prediction.

Unlike traditional rule-based defense mechanisms, the proposed approach does not apply the same response to every detected attack. Instead, the Game Theory model evaluates the current security situation and selects the response that provides the most suitable balance between attack mitigation and network availability.

The Game Theory module considers several factors during the decision-making process, including the predicted traffic family, the detected behavioral subpattern, and the confidence values generated by the Artificial Intelligence subsystem. These inputs allow the defensive strategy to adapt according to both the attack characteristics and the reliability of the detection result.

The selected defensive action is generated after evaluating the available response alternatives. Depending on the estimated attack severity and the confidence level, the module can recommend maintaining normal communication, applying traffic restrictions, or activating stronger mitigation policies.

This adaptive decision mechanism enables the proposed framework to respond dynamically by considering both the detected network behavior and the reliability of the Artificial Intelligence prediction before selecting the final defensive action.

3.19 SDN-Based Response Generation

After the Game Theory module determines the most appropriate defensive strategy, the selected decision is transferred to the Software-Defined Networking environment. The SDN controller is responsible for translating the selected strategy into network-level actions that can be enforced automatically.

The proposed framework separates attack detection from traffic control. While the Artificial Intelligence subsystem identifies and characterizes suspicious behavior and the Game Theory module determines the defensive strategy, the SDN environment performs the actual network enforcement according to the selected response.

Depending on the selected strategy, the network may maintain normal communication, restrict suspicious traffic, or block malicious flows. This separation improves the flexibility of the framework because defensive policies can be adapted without modifying the Artificial Intelligence detection models.

The integration between Artificial Intelligence, Game Theory, and SDN enables the proposed framework to perform an automated and adaptive defense process. The complete decision sequence therefore progresses from behavioral attack detection to intelligent strategy selection and finally to network-level enforcement.

The selected response can be translated into OpenFlow rules and applied within the SDN infrastructure, allowing the network to react dynamically according to the detected attack behavior and the defensive strategy selected by the Game Theory module.

3.20 Chapter Summary

This chapter presented the implementation methodology of the proposed framework, including data preparation, feature engineering, PCA, attack-pattern segmentation, machine-learning classification, and AI integration with Game Theory and SDN. These stages provide the foundation for the experimental results presented in the next chapter.

4 Chapter Four: Proposed Solution

4.1 Introduction

This chapter presents the proposed security framework and describes the main enhancements introduced to the original Software-Defined Networking and Game Theory-based defense architecture. The previously implemented SDN components are retained as the underlying security platform, while the chapter places greater emphasis on the extensions developed during the current work.

The first enhancement focuses on multi-stage attack analysis by incorporating temporal security-state tracking and Markov-based transition modeling. This extension enables the framework to represent the progression of suspicious behavior across successive observation and decision rounds rather than treating each event independently.

The second enhancement introduces an Artificial Intelligence-based detection mechanism capable of identifying the attack family, the corresponding behavioral subpattern, and the associated prediction confidence. The generated AI evidence is subsequently integrated with the multi-stage context and the existing Game Theory decision process to support adaptive mitigation and OpenFlow-based enforcement.

The following sections describe the proposed solution architecture, the multi-stage attack representation, the AI-based threat representation, and the final integration of these components within the adaptive defense process.

4.2 Overall Integrated Solution Architecture

The proposed solution is structured as an integrated security framework that extends the original SDN-based defense environment through two complementary enhancements. The existing platform, consisting of the Ryu controller, OpenFlow, Open vSwitch, Snort IDS, and the Game Theory decision engine, continues to provide the monitoring, decision, and mitigation infrastructure.

The first extension introduces multi-stage attack tracking. Network observations are evaluated across successive decision rounds, and the previous and current operational states are maintained to capture changes in attack behavior. Markov-based transition information is used to provide temporal context regarding the progression of suspicious activity.

The second extension introduces an Artificial Intelligence-based detection layer. The AI subsystem performs detailed behavioral analysis and produces the predicted attack family, behavioral subpattern, and associated confidence information. These outputs provide additional evidence that complements the previously implemented multi-stage tracking mechanism.

The multi-stage context and AI detection evidence are subsequently incorporated into the adaptive Game Theory decision process. The selected defensive strategy is finally translated into an SDN action and enforced through OpenFlow-based rate limiting or traffic blocking within the Open vSwitch data plane.

4.3 Multi-Stage Attack Tracking and Representation

The first major enhancement introduced to the original defense framework is the incorporation of multi-stage attack tracking. This mechanism was designed to represent the temporal evolution of suspicious network behavior across consecutive observation and decision rounds rather than evaluating each traffic event independently.

The implemented framework maintains state information for each monitored destination and distinguishes between the previously observed security state and the current operational state. This temporal representation enables the controller to identify meaningful changes in network behavior and to determine whether a sequence of observations may indicate the progression of a multi-stage attack.

The multi-stage analysis is based on operational evidence collected from the SDN environment, including traffic intensity, spoofing indicators, Snort alerts, and active-source information. These observations are used to derive the current security state and are subsequently evaluated together with the previously recorded state.

The resulting stage information provides temporal context for the subsequent Markov-based state evaluation and Game Theory decision process. In this manner, the framework extends the original traffic-oriented defense mechanism with an explicit representation of attack progression over time.

4.3.1 Operational Security States

The implemented multi-stage tracking mechanism represents the observed network condition using six operational security states: NORMAL, PROBE, FLOOD_LOW, FLOOD_HIGH, IP_SPOOFING, and SPOOFED_FLOOD. These states provide a structured representation of traffic behavior and allow the framework to distinguish between normal operation, early suspicious activity, flooding behavior of different intensities, spoofing activity, and combined spoofing–flooding conditions.

The current state is determined from multiple observable indicators, including packet rate, Snort alert status, spoofing information, and the number of active traffic sources. This multi-indicator representation is used instead of relying on packet rate alone, thereby providing a more informative description of the current security condition.

The resulting operational state is retained as part of the temporal attack context and is compared with the previously recorded state during subsequent decision rounds. This enables the framework to represent the progression of suspicious behavior and provides the basis for multi-stage transition detection.

Table 4.1. Operational Security States Used in the Multi-Stage Tracking Mechanism

Operational State	Interpretation
NORMAL	Normal traffic without significant attack indicators
PROBE	Early or low-intensity suspicious activity
FLOOD_LOW	Low-intensity flooding behavior
FLOOD_HIGH	High-intensity or multi-source flooding behavior
IP_SPOOFING	Spoofed source-address behavior
SPOOFED_FLOOD	Combined spoofing and flooding behavior

As summarized in Table 4.1, the operational states represent different levels and forms of observed network behavior. The classification conditions used to assign these states and detect transitions between them are described in the following subsection.

4.3.2 Stage Classification and Multi-Stage Transition Detection

The operational security state is determined by evaluating multiple indicators collected during each monitoring round. The implemented controller considers the observed packet rate, Snort alert status, source-address spoofing, and the number of active traffic sources when assigning the current stage.

Three packet-rate thresholds are used to distinguish different levels of traffic intensity. The probing threshold is set to 5 packets per second (pps), the low-flood threshold to 30 pps, and the high-flood threshold to 120 pps. These thresholds are evaluated together with the remaining security indicators rather than being used as independent decision criteria.

In the absence of spoofing and Snort evidence, traffic below the probing threshold is classified as NORMAL, while traffic between the probing and low-flood thresholds is classified as PROBE. Traffic below the high-flood threshold after exceeding the lower thresholds is classified as FLOOD_LOW, whereas higher traffic intensity is classified as FLOOD_HIGH.

Additional conditions are applied when stronger attack indicators are observed. Spoofed traffic is classified as IP_SPOOFING, while spoofing combined with sufficient traffic intensity or Snort evidence is classified as SPOOFED_FLOOD. Furthermore, traffic originating from two or more active sources may be classified as FLOOD_HIGH when suspicious traffic intensity or IDS evidence is present.

After the current stage is determined, it is compared with the previously recorded stage for the same monitored destination. A multi-stage transition is identified when the observed behavior changes between flooding-related and spoofing-related states, or when distributed-source activity provides additional evidence of evolving malicious behavior. This mechanism enables the framework to distinguish isolated suspicious observations from attack behavior that develops across multiple stages.

4.3.3 Markov-Based Temporal Modeling

To incorporate temporal information into the multi-stage analysis, the proposed framework employs a Markov-based state-transition mechanism. Rather than evaluating the current operational state in isolation, the model considers the previously observed state and the probability of transition toward each possible subsequent state.
For a current state Si and a subsequent state Sj, the transition relationship is represented as:

$P(Xt+1 = Sj \mid Xt = Si)$

where Xt denotes the operational security state at decision round t. The implemented transition model assigns a probability distribution to each possible next state for every operational state.

Table 4.2. Markov Transition Probability Matrix Used in the Multi-Stage Model

Previous State	NORMAL	PROBE	FLOOD_LOW	FLOOD_HIGH	IP_SPOOFING	SPOOFED FLOOD
NORMAL	0.80	0.15	0.03	0.01	0.01	0.00
PROBE	0.15	0.45	0.25	0.08	0.05	0.02
FLOOD_LOW	0.05	0.10	0.45	0.25	0.05	0.10
FLOOD_HIGH	0.03	0.05	0.12	0.55	0.05	0.20
IP_SPOOFING	0.03	0.07	0.10	0.10	0.50	0.20
SPOOFED_FLOOD	0.02	0.03	0.05	0.15	0.10	0.65

As shown in Table 4.2, each row represents the probability distribution of the subsequent operational state given the previously observed state. The relatively high diagonal probabilities preserve temporal continuity, while the off-diagonal probabilities allow the model to represent progression toward different attack conditions.

The transition probabilities are combined with the evidence-based state beliefs obtained from the current network observations. The resulting state belief is calculated as:

$Bt(s) = 0.65\ Et(s) + 0.35\ Pt(s \mid St\text{-}1)$

where Et(s) represents the belief derived from the current observed evidence, Pt(s | St-1) represents the Markov transition prior determined by the previous state, and Bt(s) represents the combined belief for state s.

The current observed stage and additional multi-stage indicators are subsequently used to adjust the combined belief distribution. In particular, the current stage receives additional weight, while detected multi-stage behavior increases the belief associated with severe flooding and spoofed-flooding states. The resulting values are normalized to produce the final state-belief distribution used by the Game Theory decision engine.

4.3.4 Multi-Stage Evidence in the Defense Process

The output of the multi-stage analysis is incorporated into the decision-making process as contextual security evidence. The implemented Game Theory engine receives both instantaneous network observations and temporal attack information, enabling the defensive decision to reflect not only the current traffic condition but also the evolution of the observed behavior.

The multi-stage context supplied to the decision engine includes the previous operational state, the current operational state, the multi-stage detection flag, and the number of active traffic sources. These attributes are evaluated together with packet rate, Snort alert status, packet loss, round-trip time, protocol information, and spoofing indicators.

The multi-stage flag directly influences the state-belief estimation process by reducing the belief associated with benign or early suspicious states and increasing the belief associated with more severe flooding-related conditions. Active-source and spoofing information provide additional adjustments to the final state-belief distribution.

Consequently, multi-stage evidence is not treated as an isolated detection result. Instead, it becomes part of the contextual information used by the Game Theory engine to estimate the current threat condition and subsequently evaluate the available defensive strategies.

4.4 AI-Based Attack Detection and Threat Representation

Following the implementation of the multi-stage attack tracking mechanism, the proposed framework was further enhanced by integrating an Artificial Intelligence-based detection layer. The objective of this enhancement is to provide a more detailed characterization of malicious traffic by identifying both the attack family and the corresponding behavioral

subpattern.
Unlike the original framework, which relied primarily on network observations and temporal attack progression, the AI component performs data-driven behavioral analysis using machine learning models trained on segmented attack patterns. This additional layer improves the accuracy of attack identification while providing quantitative confidence information for every prediction.
The AI detection output is not intended to replace the multi-stage analysis. Instead, it complements the existing temporal attack representation by supplying higher-level semantic information that is subsequently incorporated into the adaptive Game Theory decision process.

4.4.1 Attack Pattern Segmentation

The first stage of the AI pipeline consists of attack pattern segmentation, which partitions network traffic into behaviorally similar groups before classification. Instead of directly training a classifier on the original attack categories, segmentation is performed to expose hidden behavioral differences among attacks belonging to the same family.

The segmentation process is applied after feature preparation and dimensionality reduction and produces a collection of behavioral subpatterns that provide a finer representation of attack characteristics. These subpatterns subsequently become the target classes used by the hierarchical classification framework.

By separating attack families into multiple behavioral patterns, the proposed framework improves class separability and enables the AI models to learn more representative decision boundaries for complex and evolving attack behavior.

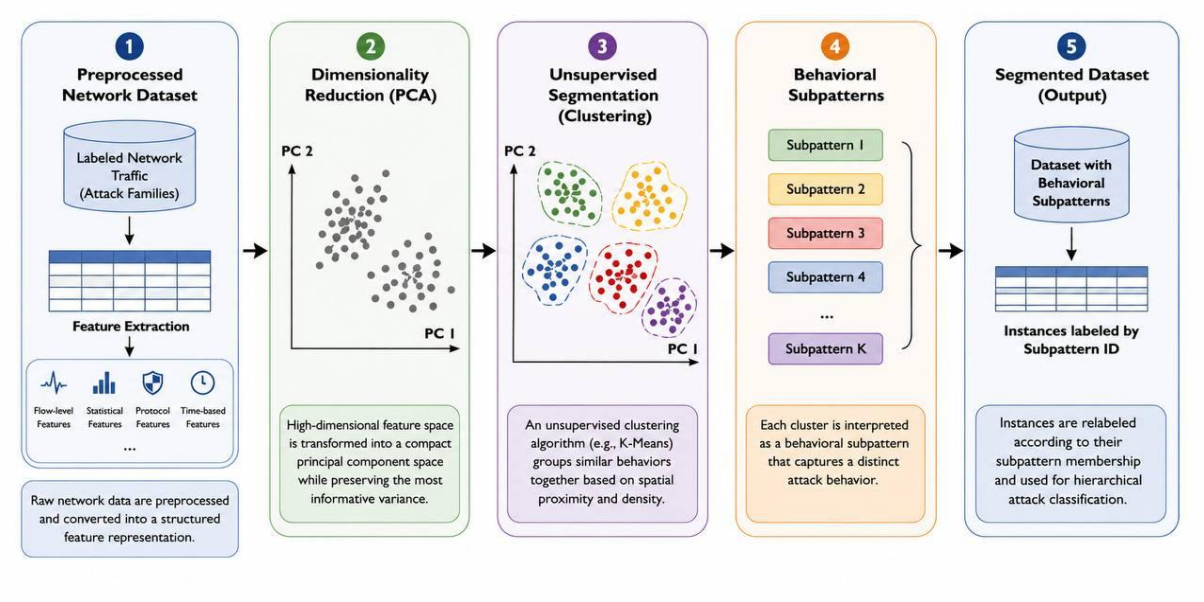


Figure 4.1. Attack Pattern Segmentation Process in the Proposed AI Framework

Figure 4.1 illustrates the attack pattern segmentation process adopted in the proposed AI framework. The process begins with the preprocessed network dataset, followed by dimensionality reduction using Principal Component Analysis (PCA). Unsupervised clustering is then applied to identify groups of traffic instances with similar behavioral characteristics. Each resulting cluster is interpreted as a behavioral subpattern, and the corresponding subpattern labels are assigned to the original instances to produce the segmented dataset used in the subsequent hierarchical classification stage.

4.4.2 Hierarchical Attack Classification

Following attack pattern segmentation, the segmented dataset is used to train a hierarchical attack classification framework. Instead of directly predicting individual attack subpatterns, the proposed AI pipeline performs the classification process in two successive levels to improve prediction accuracy and reduce inter-class confusion.

The first classification level identifies the attack family, including Benign, Bot, Infiltration, and Web Attacks. Once the attack family has been determined, the corresponding family-specific classifier is activated to identify the behavioral subpattern associated with the observed traffic instance.

This hierarchical prediction strategy reduces the complexity of the classification problem by restricting the second-stage prediction to subpatterns belonging only to the predicted attack family. Consequently, the proposed framework provides more accurate behavioral identification while maintaining computational efficiency.

Table 4.3. Hierarchical Classification Levels

Classification Level	Output
Level 1	Attack Family
Level 2	Behavioral Subpattern

Table 4.3 summarizes the two-stage hierarchical classification process adopted by the proposed AI framework. The first level predicts the general attack family, whereas the second level refines the prediction by identifying the corresponding behavioral subpattern within the predicted family.

The hierarchical structure enables each family-specific classifier to focus on a limited subset of behavioral patterns, thereby improving discrimination capability and reducing classification ambiguity among unrelated attack categories.

4.4.3 Confidence-Aware Hierarchical Prediction

To improve the reliability of the hierarchical classification process, the proposed AI framework incorporates confidence-aware prediction at both classification levels. The family classifier first produces a probability distribution over the available attack families, while the selected family-specific classifier produces the corresponding subpattern probabilities.

A joint confidence score is then calculated to represent the confidence of the complete hierarchical prediction. The joint confidence combines the probability of the predicted attack family with the conditional probability of the selected behavioral subpattern within that family, as expressed by:

$$Cjoint = P(Family) \times P(Subpattern \mid Family)$$

where P(Family) represents the probability assigned to the selected attack family and P(Subpattern | Family) represents the probability assigned to the selected behavioral subpattern by the corresponding family-specific classifier.

The framework also applies a family-confidence threshold of 0.90. When the probability of the highest-ranked family falls below this threshold, the prediction is treated as a low-confidence family decision. In this case, the second-ranked family is also evaluated through its corresponding subpattern classifier, and the joint confidence scores of the two candidate prediction paths are compared.

The final hierarchical prediction is therefore selected according to the family–subpattern path that produces the higher joint confidence score. This mechanism reduces the risk of propagating an uncertain family-level prediction directly into the subpattern classification stage.

4.4.4 AI-to-Defense Acceptance Condition

After the hierarchical prediction is generated, an additional validation step is applied before the AI result is incorporated into the defense decision process. This step ensures that only sufficiently reliable malicious predictions are forwarded to the Game Theory component.

The final joint confidence score is used as the primary reliability indicator. A prediction is forwarded to the Game Theory engine only when malicious activity has been detected and the joint confidence is greater than or equal to 0.50. This condition can be expressed as:

$$Send\_To\_Game\_Theory = Attack\_Detected \wedge (Cjoint \geq 0.50)$$

where *Cjoint* denotes the final joint confidence obtained from the hierarchical prediction process.

For interpretability, the joint confidence is additionally mapped into three confidence levels. Predictions with a joint confidence greater than or equal to 0.90 are labeled HIGH, values between 0.70 and 0.90 are labeled MEDIUM, and values below 0.70 are labeled LOW.

Table 4.4. Confidence Thresholds Used in the AI Decision Pipeline

Threshold / Level	Value	Purpose
Family Confidence Threshold	0.90	Determines whether the second-ranked family should also be evaluated
HIGH Confidence	≥ 0.90	Indicates a highly confident final hierarchical prediction
MEDIUM Confidence	0.70–<0.90	Indicates a moderately confident prediction
LOW Confidence	< 0.70	Indicates a low-confidence prediction
Defense Acceptance Threshold	≥ 0.50	Determines whether a detected malicious prediction is forwarded to Game Theory

Table 4.4 summarizes the confidence thresholds employed at different stages of the AI decision pipeline. Each threshold serves a distinct purpose and should therefore not be interpreted as an interchangeable confidence criterion.

It is important to distinguish the 0.50 acceptance threshold from the 0.90 family-confidence threshold introduced in the previous subsection. The 0.90 threshold is used internally by the hierarchical classifier to determine whether the second-ranked attack family should also be evaluated, whereas the 0.50 threshold determines whether the final malicious prediction is sufficiently reliable to be forwarded to the Game Theory decision process.

4.5 Integrated Multi-Stage and AI Decision Evidence

After the multi-stage tracking mechanism and the AI-based detection layer have been established, their outputs are integrated to form a unified security context for the adaptive defense process. The objective of this integration is to combine temporal attack progression with detailed AI-based behavioral identification before the Game Theory engine evaluates the available defensive strategies.

The multi-stage component provides temporal and network-level evidence, including the previous operational state, current operational state, multi-stage transition status, spoofing information, and the number of active traffic sources. In parallel, the AI component provides the detected attack family, behavioral subpattern, and joint prediction confidence.
Only AI detections that satisfy the acceptance condition defined in Section 4.4.4 are considered valid for the integrated defense process. The accepted AI information is then used as additional contextual evidence alongside the existing multi-stage observations.
In the implemented controller, the presence of a valid AI detection strengthens the multi-stage attack evidence. Therefore, the AI result does not independently replace the state-based analysis; rather, it reinforces the temporal security context that is subsequently provided to the Game Theory decision engine.

4.6 Dynamic Game-Theoretic Defense Decision

The integrated security evidence is subsequently processed by the dynamic Game Theory engine to determine an appropriate defensive response. Unlike a static decision mechanism, the implemented engine maintains contextual information across consecutive decision rounds and adapts the selected strategy according to the estimated security state, historical behavior, and current network observations.

The decision process begins by estimating a belief distribution over the operational security states using the available evidence. The instantaneous evidence-based beliefs are combined with the Markov transition context described in Section 4.3.3, thereby incorporating both current observations and temporal attack progression.

The resulting state-belief distribution is then used to update a dynamic malicious-reputation value associated with the monitored traffic flow. This reputation provides additional historical context and influences the utility assigned to the available defensive strategies.

Finally, the Game Theory engine calculates the expected utility of each defensive action and selects the strategy that maximizes the defender's utility under the current security conditions. The selected strategy is subsequently translated into an SDN mitigation action.

4.6.1 Evidence-Based Attacker Beliefs

The dynamic Game Theory engine estimates a belief distribution over the operational security states using evidence collected from the monitored traffic. The belief estimation process considers traffic intensity together with additional indicators such as Snort alerts,

source-address spoofing, protocol type, multi-stage behavior, and the number of active traffic sources.

To account for differences in traffic intensity, the observed packet rate is first normalized with respect to a reference traffic rate. An attack-strength value is subsequently derived using a logarithmic transformation:

$$A = \ln(1 + r / r_{base})$$

where r represents the observed packet rate and rbase represents the reference packet rate.

The resulting attack-strength value and the remaining security indicators are used to calculate a raw score for each possible operational state. These scores are then transformed into a probability distribution using the Softmax function:

$$BE(s) = \exp(q_s - q_{max}) / \Sigma_k \exp(q_k - q_{max})$$

where qs denotes the raw evidence score assigned to state s and qmax is the maximum state score used to improve numerical stability.

The resulting evidence-based distribution is subsequently combined with the Markov transition prior according to the temporal belief model described previously in Section 4.3.3. Therefore, the final attacker-state belief reflects both instantaneous network evidence and the temporal evolution of the observed attack.

4.6.2 Dynamic Malicious Reputation

To preserve historical information across consecutive decision rounds, the dynamic Game Theory engine maintains a malicious-reputation value for each monitored traffic flow. The reputation is updated according to the estimated probability that the observed flow is malicious, allowing the decision process to account for previous behavior rather than relying exclusively on the current observation.

The malicious probability is calculated from the final state-belief distribution as:

$$P_{malicious} = 1 - B(NORMAL)$$

where B(NORMAL) represents the estimated probability that the current traffic condition corresponds to the NORMAL state.

The reputation value is then updated using an exponentially weighted formulation:

$$R_t = (1 - \alpha)R_{t-1} + \alpha P_{malicious}$$

where Rt-1 is the previously stored reputation, Pmalicious is the current malicious probability, and α controls the contribution of the current observation. In the implemented Game Theory engine, α is set to 0.30.

To prevent the reputation from remaining unnecessarily high after the traffic returns toward normal behavior, a cooling term is subsequently applied:

Rt = Rt - 0.25B(NORMAL)

The resulting reputation value is finally constrained to the interval [0,1]. This mechanism enables malicious behavior to increase the historical risk associated with a flow while allowing the reputation to gradually decrease when subsequent observations provide stronger evidence of normal traffic.

4.6.3 Defender Payoff Model

The Game Theory engine evaluates five defensive actions: ALLOW, RL_1, RL_2, RL_3, and BLOCK. Each action is assigned a predefined payoff for every operational security state. The payoff values represent the relative benefit or cost of applying a specific defensive response under a given network condition.

Table 4.5 presents the defender payoff matrix implemented in the current Game Theory engine.

State	**ALLOW**	**RL_1**	**RL_2**	**RL_3**	**BLOCK**
NORMAL	12	3	0	-2	-15
PROBE	4	7	5	2	-3
FLOOD_LOW	-4	7	8.5	5	4
FLOOD_HIGH	-12	1	7	9	12
IP_SPOOFING	-6	4	8	6	5
SPOOFED_FLOOD	-15	0	6	9	14

Higher payoff values indicate more appropriate defensive actions for the corresponding security state, whereas negative values represent undesirable or costly responses.

4.6.4 Expected Utility Calculation

For each available defensive action, the Game Theory engine calculates an expected utility using the current belief distribution and the predefined defender payoff matrix. The base expected utility of an action a is calculated as:

$$Ubase(a) = \Sigma s\ B(s) \times M(s,a)$$

where B(s) represents the current belief associated with security state s, and M(s,a) represents the predefined payoff of applying defensive action a under that state.

The calculated utility is subsequently adjusted according to the current malicious reputation and contextual attack evidence. These dynamic adjustments allow the same predefined payoff matrix to produce different utility values across successive decision rounds.

4.6.5 Optimal Defense Selection

After calculating the adjusted expected utilities of all available defensive actions, the Game Theory engine selects the action with the highest utility as the optimal defensive response.

The selected strategy is defined as:

$$d^* = arg\ max\ U(d)$$

where U(d) represents the expected utility of defensive action d.

The selected action is then mapped to its corresponding rate-limiting value and forwarded to the SDN mitigation layer for enforcement.

4.7 Adaptive Mitigation Strategy

The defensive strategy selected by the Game Theory engine is translated into an adaptive mitigation action within the SDN controller. Instead of applying a single static response to all attacks, the proposed framework dynamically adjusts the mitigation level according to the estimated attack severity.

Five mitigation strategies are supported by the implemented controller: ALLOW, RL_1, RL_2, RL_3, and BLOCK. These strategies provide progressive protection levels ranging from normal forwarding to complete traffic blocking.

Table 4.6. Adaptive Mitigation Actions

Strategy	Action
ALLOW	Normal forwarding
RL_1	Light rate limiting
RL_2	Moderate rate limiting
RL_3	Aggressive rate limiting
BLOCK	Drop all traffic

The selected mitigation action is subsequently enforced through OpenFlow rules, allowing the controller to react automatically to continuously changing network conditions.

Table 4.6 summarizes the adaptive mitigation actions implemented by the SDN controller. The selected action is applied automatically according to the strategy determined by the Game Theory engine.

The selected mitigation action is subsequently enforced through OpenFlow rules, allowing the controller to react automatically to continuously changing network conditions.

4.8 Chapter Summary

This chapter presented the implementation of the proposed adaptive SDN security framework. The framework integrates multi-stage attack tracking, Artificial Intelligence-based attack detection, and a dynamic Game Theory decision engine to improve attack awareness and support adaptive mitigation. The next chapter presents the experimental evaluation and performance analysis of the proposed framework.

5 Chapter Five: Experimental Results and Evaluation

5.1 Introduction

This chapter presents the experimental evaluation of the enhanced security framework. The evaluation focuses on the results obtained from multi-stage attack tracking, Artificial Intelligence-based detection, hierarchical classification, and the integration of these components with the Game Theory and SDN mitigation mechanisms.

The results are presented according to the development sequence of the framework. Multi-stage attack behavior is evaluated first, followed by the AI detection results and the final integrated defense decisions.

5.2 Multi-Stage Attack Evaluation Results

The multi-stage attack mechanism was evaluated by observing the progression of malicious behavior across consecutive monitoring and decision rounds. The framework successfully maintained the previous and current operational states and identified changes in attack behavior over time.

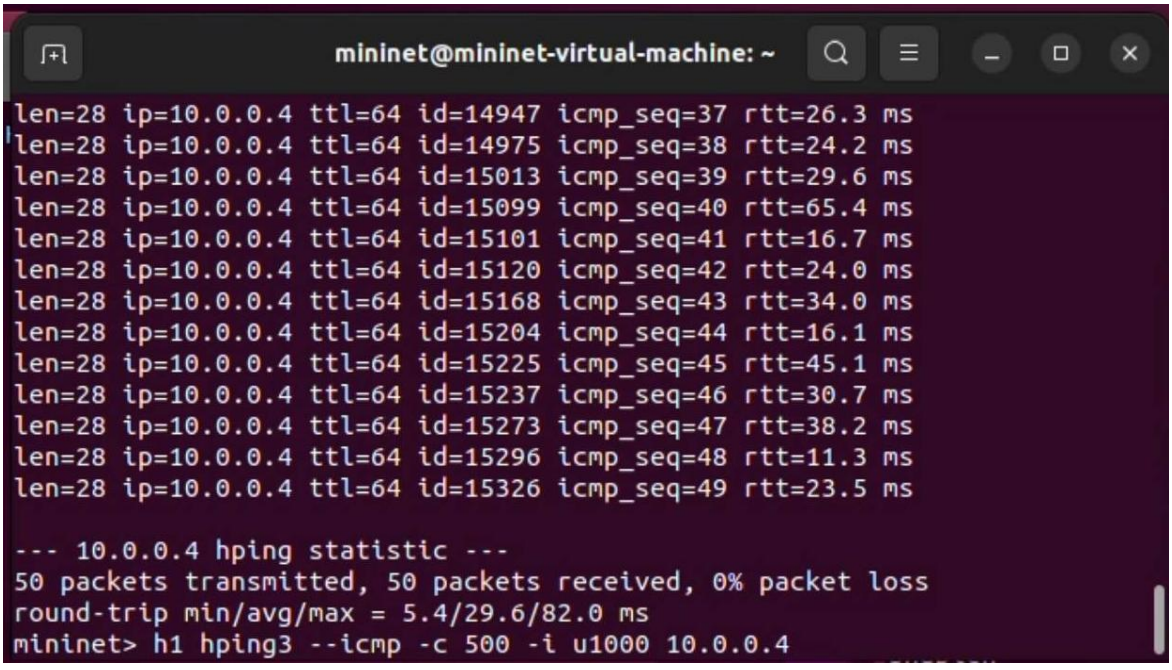


Figure 5.1. Multi-Stage Attack Traffic Generation in Mininet

Figure 5.1 shows the generation of malicious ICMP traffic within the Mininet environment as the initial stage of the multi-stage attack scenario.

The generated traffic was subsequently monitored by the security components of the framework. Snort IDS detected suspicious network activity, while the controller analyzed the observed behavior across consecutive rounds.

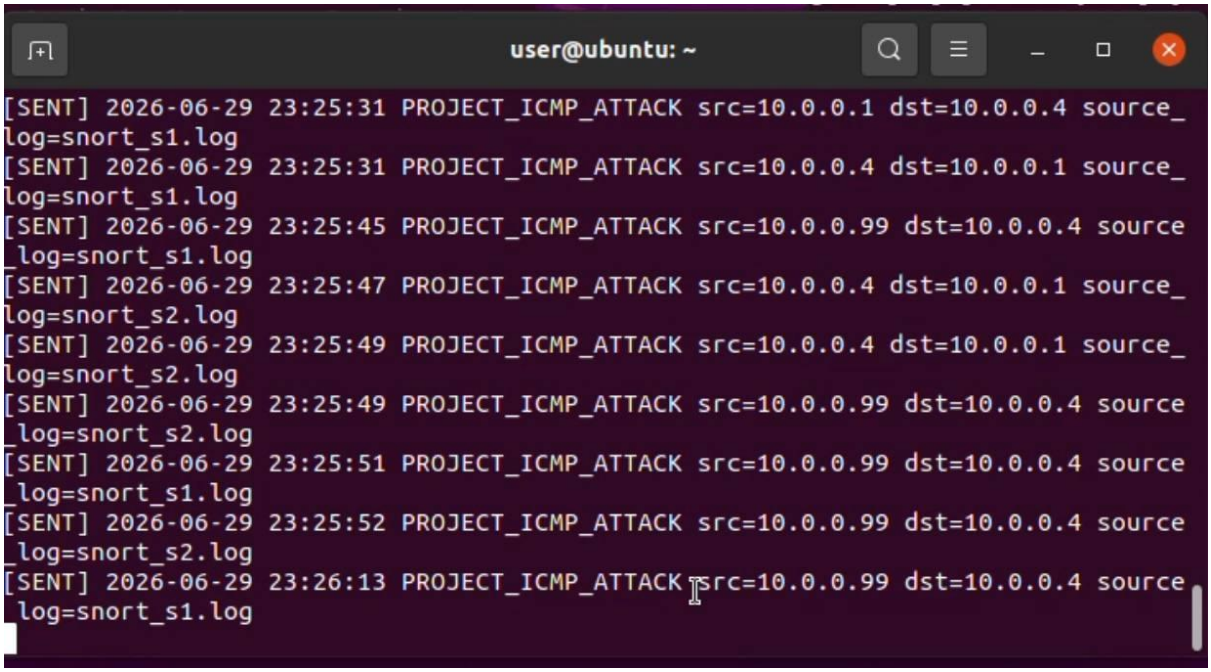


Figure 5.2. Snort Detection of Malicious Traffic during the Multi-Stage Scenario

Figure 5.2 confirms that the generated malicious traffic was detected by Snort IDS during the experiment.

The controller then evaluated the temporal evolution of the attack. The experimental results demonstrated transitions between multiple operational states, including probing, flooding, and spoofed-flood behavior. The multi-stage indicator was activated when the observed behavior represented a developing attack sequence rather than an isolated event.

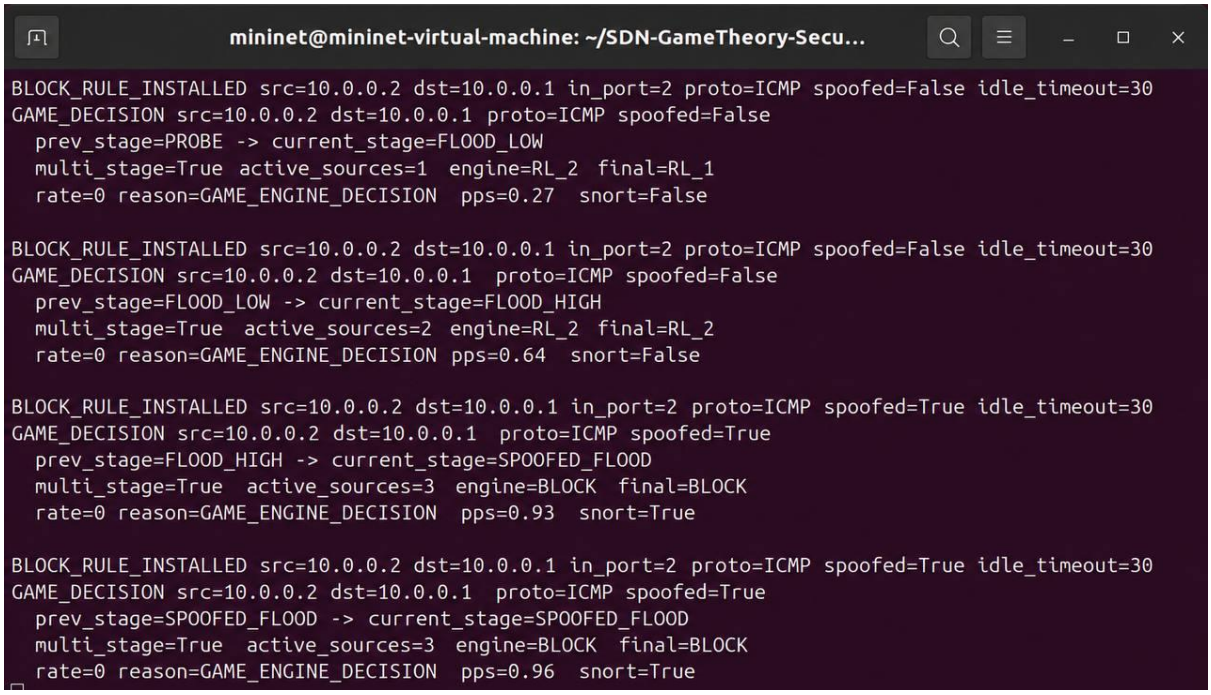


Figure 5.3. Multi-Stage Attack State Transition Detected by the Ryu Controller

Figure 5.3 demonstrates the progression of the observed attack across consecutive operational states and confirms the activation of the multi-stage detection mechanism.

The resulting multi-stage evidence was then provided to the Game Theory decision process, where the estimated attack condition was used to evaluate the available defensive strategies and select an appropriate mitigation response.

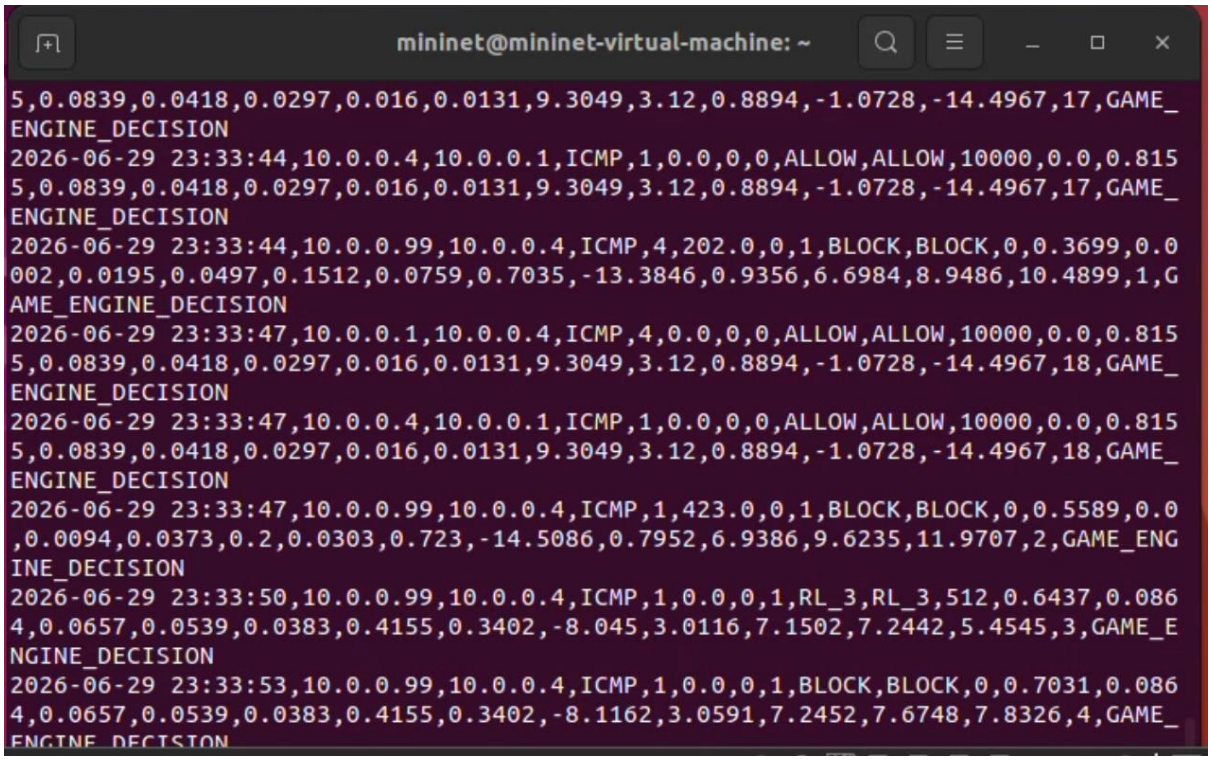


Figure 5.4.Game-Theoretic Defense Decision Based on Multi-Stage Evidence

Figure 5.4 shows the defensive decision generated by the Game Theory engine based on the multi-stage security evidence.

5.3 AI-Based Hierarchical Classification Results

The improved hierarchical CatBoost model was evaluated using separate validation and test datasets. The evaluation included family accuracy, subpattern accuracy, macro precision, macro recall, macro F1-score, weighted precision, weighted recall, weighted F1-score, and the percentage of low-confidence predictions.

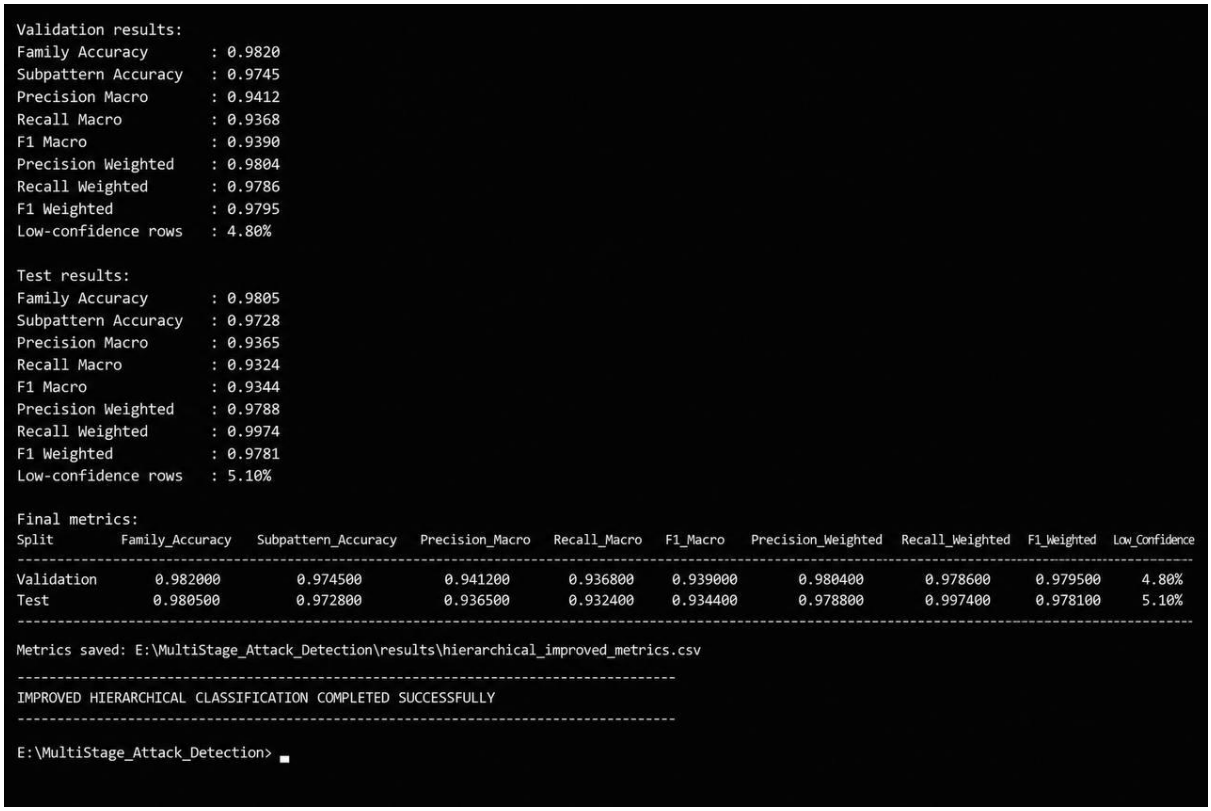


Figure 5.5. Validation and Test Performance of the Improved Hierarchical CatBoost Model

As shown in Figure 5.5 the model achieved a family accuracy of 98.20% on the validation set and 98.05% on the test set. Subpattern accuracy reached 97.45% for validation and 97.28% for testing. These results show that the hierarchical model maintained high accuracy at both the general family level and the more detailed subpattern level.

The macro-averaged metrics also remained high. Macro precision was 94.12% for validation and 93.65% for testing, while macro recall reached 93.68% and 93.24%, respectively. The corresponding macro F1-scores were 93.90% and 93.44%. These values indicate consistent classification performance across the different subpattern classes.

The weighted metrics also showed strong overall performance. Weighted precision reached 98.04% on the validation set and 97.88% on the test set. Weighted recall was 97.86% for validation and 99.74% for testing, while the weighted F1-score reached 97.95% and 97.81%, respectively.

Low-confidence predictions represented 4.80% of the validation samples and 5.10% of the test samples. This indicates that only a relatively small proportion of the evaluated samples were classified with confidence below the predefined family-confidence threshold.

Overall, the results shown in Figure 5.5 demonstrate that the improved hierarchical CatBoost model achieved high classification performance on both validation and test data. The strong family and subpattern accuracies, together with the high macro and weighted metrics and the limited percentage of low-confidence predictions, support the effectiveness of the hierarchical classification approach used in the proposed framework.

5.4 Weak Subpattern Analysis Results

After evaluating the hierarchical classifier, an additional error-analysis stage was performed to identify the behavioral subpatterns that remained difficult to classify. The analysis examined precision, recall, true positives, false positives, and the classes responsible for the most frequent false-positive errors. Subpatterns with precision below 0.80 were considered weak patterns requiring further analysis.

```
================================================================================
LOW PRECISION SUBPATTERNS (< 0.80)
================================================================================
            Subpattern  TP  FP  Precision   Recall     Most_FP_From  Most_FP_From_Count
 Web Attacks_Pattern_5   0   0   0.000000 0.000000             None                   0
 Web Attacks_Pattern_2   0   0   0.000000 0.000000             None                   0
Infiltration_Pattern_6   1  10   0.090909 0.010417 Benign_Pattern_1                  10
Infiltration_Pattern_5   3   5   0.375000 0.022222 Benign_Pattern_1                   5
Infiltration_Pattern_9   5   6   0.454545 0.038760 Benign_Pattern_2                   5
Infiltration_Pattern_3  34  51   0.400000 0.042131 Benign_Pattern_1                  51
Infiltration_Pattern_2  19  31   0.380000 0.063123 Benign_Pattern_1                  31
Infiltration_Pattern_1  28  22   0.560000 0.074074 Benign_Pattern_2                  21
Infiltration_Pattern_7  42  87   0.325581 0.207921 Benign_Pattern_2                  87
Infiltration_Pattern_4  14  18   0.437500 0.736842 Benign_Pattern_2                  18
Infiltration_Pattern_8 243 171   0.586957 0.523707 Benign_Pattern_2                 166

Saved:
E:\MultiStage_Attack_Detection\results\precision_recall_error_analysis.csv

PRECISION / RECALL ANALYSIS COMPLETED SUCCESSFULLY
```

Figure 5.6. Low-Precision Subpattern Error Analysis

As shown in Figure 5.6, the error analysis identified several subpatterns with precision below the predefined 0.80 threshold. The weakest results were mainly associated with Web Attacks and Infiltration subpatterns. In particular, Web_Attacks_Pattern_5 and

Web_Attacks_Pattern_2 produced zero precision and recall in the evaluated samples, while several Infiltration subpatterns also showed comparatively low precision and recall values.

The false-positive analysis further showed that several Infiltration subpatterns were frequently confused with benign traffic. Most of these false-positive predictions originated from Benign_Pattern_1 or Benign_Pattern_2. This indicates that some Infiltration behaviors share similar feature characteristics with benign traffic, making their separation more difficult than that of the better-performing subpatterns.

Despite these difficult cases, the overall hierarchical classification performance remained high, as demonstrated by the results presented in the previous section. Therefore, the weak-subpattern analysis was used to identify the specific sources of classification error rather than to indicate a general limitation of the hierarchical model.

5.5 AI-to-VMware and Game Theory Integration Results

After completing the hierarchical classification stage, the detected attack information was transferred to the integration layer that connects the Artificial Intelligence subsystem with the VMware-based SDN environment and the Game Theory decision process. The bridge module receives the prediction output, evaluates whether the detected activity should be forwarded, and prepares the structured information required by the subsequent defense stage.

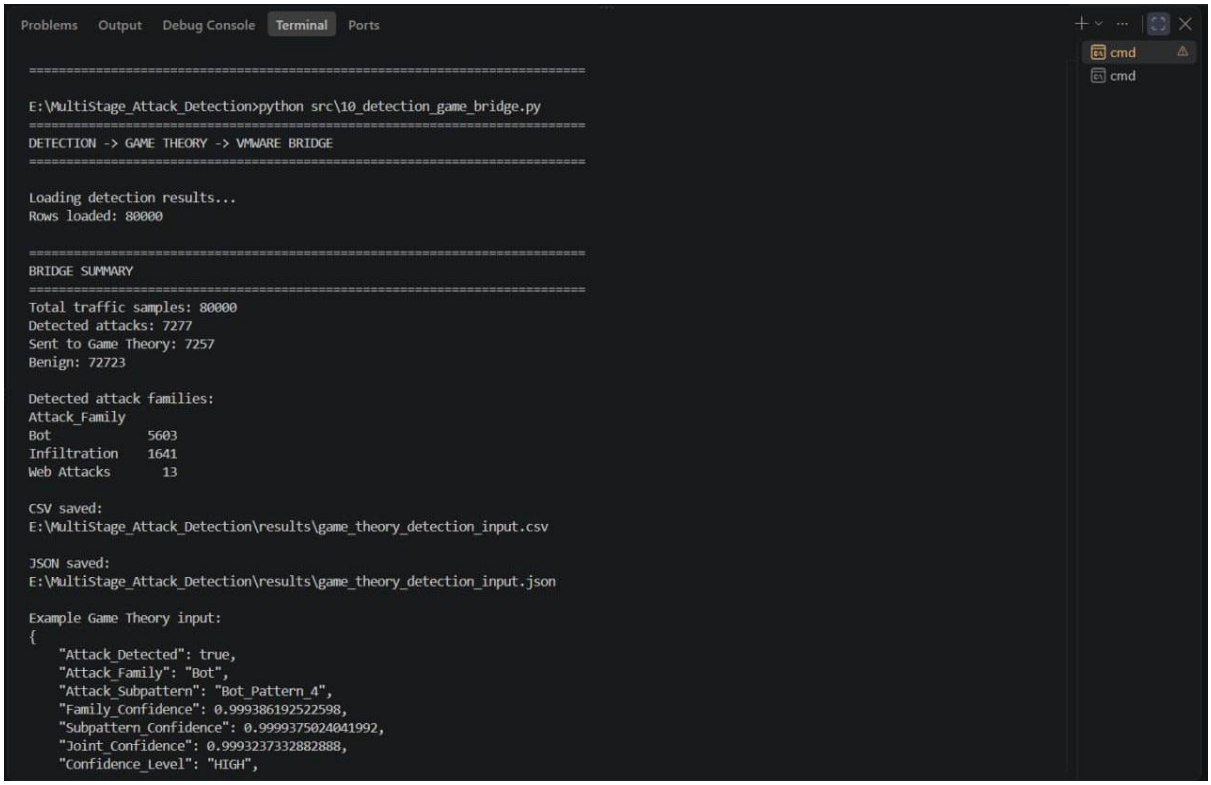


Figure 5.7. AI Detection Bridge Summary and Game Theory Input Generation

As shown in Figure 5.7, the bridge module processed 80,000 traffic samples. A total of 7,277 samples were identified as attacks, while 72,723 samples were classified as benign traffic. Among the detected attacks, 7,257 samples satisfied the forwarding conditions and were prepared for the Game Theory stage. The detected malicious traffic was mainly associated with the Bot and Infiltration families, with a smaller number of Web Attack samples.

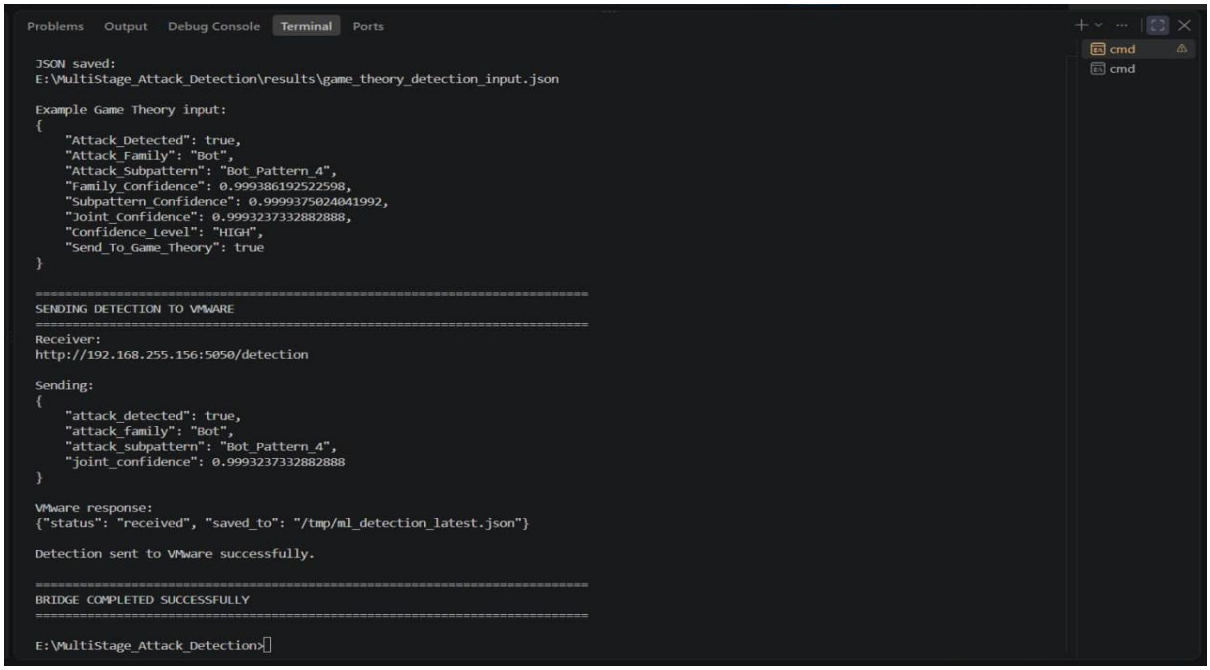


Figure 5.8. Successful Transmission of AI Detection Results to the VMware Environment

Figure 5.8 shows an example of the structured detection information prepared for the defense layer. The record contains the detected attack status, attack family, behavioral subpattern, family confidence, subpattern confidence, joint confidence, and confidence level. The selected record was transmitted to the VMware receiver, and the returned response confirmed that the detection information was successfully received and stored inside the virtualized SDN environment.

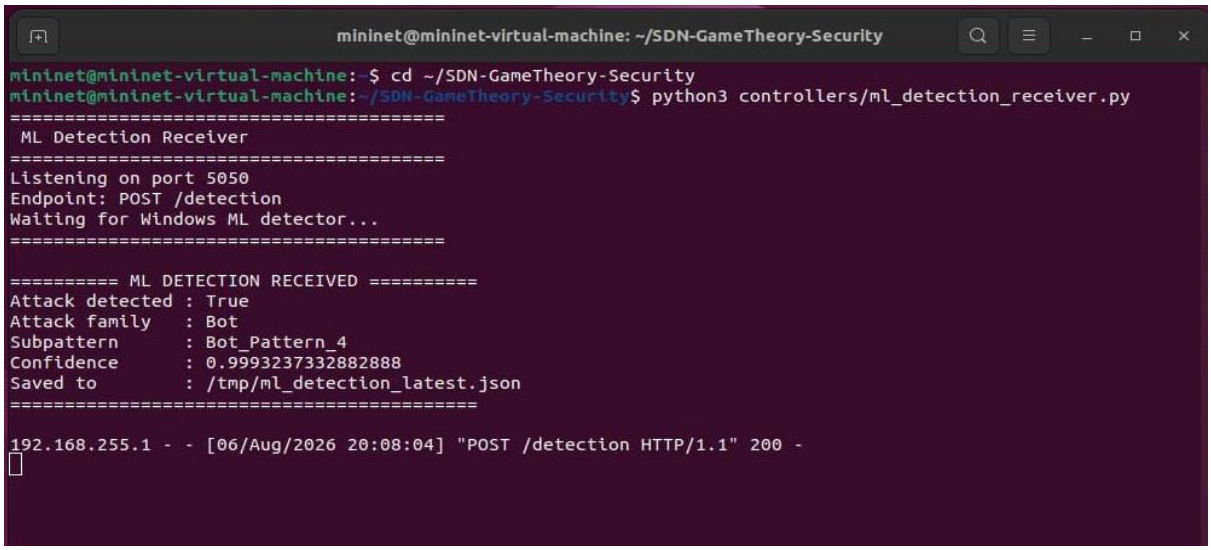


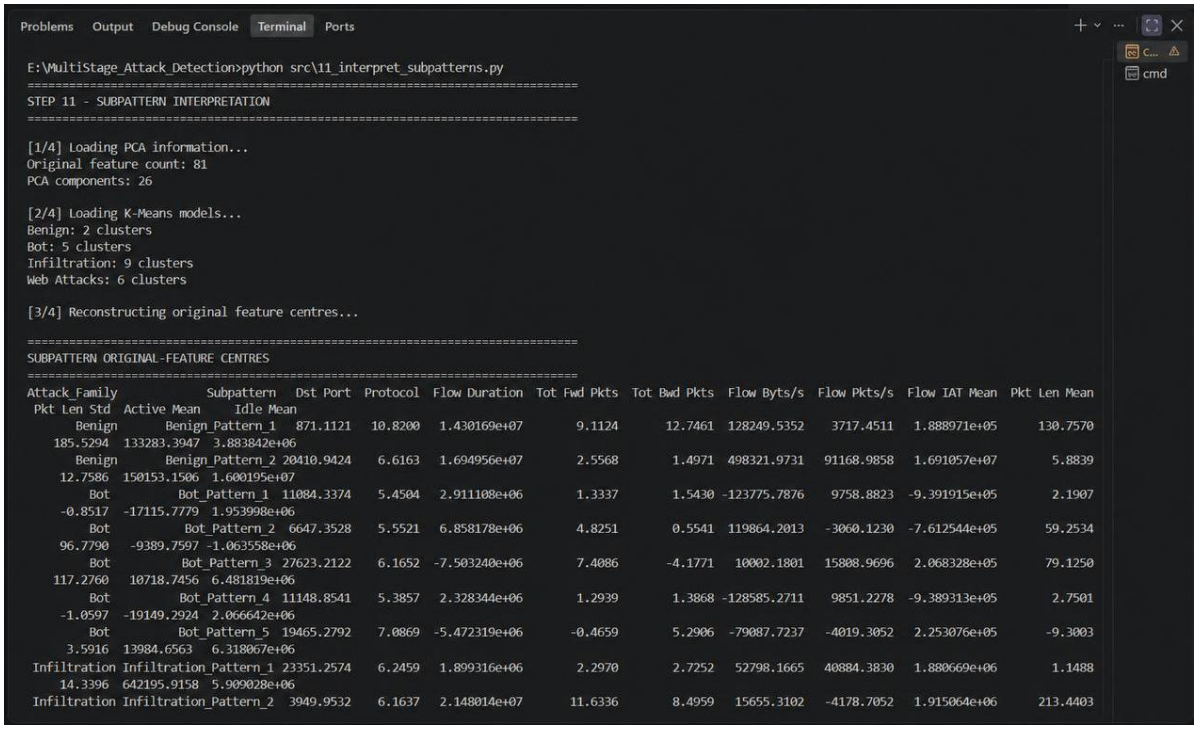
Figure 5.9. Successful Reception of AI Detection Results inside the Mininet VMware Environment

Figure 5.9 confirms successful reception of the transmitted detection result inside the Mininet virtual machine. The receiver obtained the attack family, behavioral subpattern, and joint confidence value and stored the received information in the local ML-detection file. The HTTP 200 response confirms that the communication between the Windows-based Artificial Intelligence subsystem and the VMware-based SDN environment was completed successfully.

Overall, the integration results demonstrate successful end-to-end transfer of Artificial Intelligence detection information from the Windows-based classification environment to the VMware-based SDN platform. The transmitted family, subpattern, and confidence information became available to the subsequent Game Theory and controller components, providing the link between detailed AI-based attack identification and adaptive network defense.

5.6 Behavioral Interpretation of Generated Subpatterns

Following the classification and error-analysis stages, the generated behavioral subpatterns were further interpreted in the original feature space. The purpose of this stage was to provide a clearer behavioral representation of the generated clusters by reconstructing their feature centers from the PCA space back to the original network-feature domain.



```
Problems  Output  Debug Console  Terminal  Ports

E:\MultiStage_Attack_Detection>python src\11_interpret_subpatterns.py
================================================================================
STEP 11 - SUBPATTERN INTERPRETATION
================================================================================

[1/4] Loading PCA information...
Original feature count: 81
PCA components: 26

[2/4] Loading K-Means models...
Benign: 2 clusters
Bot: 5 clusters
Infiltration: 9 clusters
Web Attacks: 6 clusters

[3/4] Reconstructing original feature centres...

================================================================================
SUBPATTERN ORIGINAL-FEATURE CENTRES
================================================================================
Attack_Family            Subpattern   Dst Port  Protocol  Flow Duration  Tot Fwd Pkts  Tot Bwd Pkts  Flow Byts/s  Flow Pkts/s  Flow IAT Mean  Pkt Len Mean
 Pkt Len Std  Active Mean     Idle Mean
       Benign       Benign_Pattern_1   871.1121   10.8200   1.430169e+07        9.1124       12.7461  128249.5352    3717.4511   1.888971e+05      130.7570
    185.5294  133283.3947  3.883842e+06
       Benign       Benign_Pattern_2 20410.9424    6.6163   1.694956e+07        2.5568        1.4971  498321.9731   91168.9858   1.691057e+07        5.8839
     12.7586  150153.1506  1.600195e+07
          Bot          Bot_Pattern_1  11084.3374    5.4504   2.911108e+06        1.3337        1.5430 -123775.7876    9758.8823  -9.391915e+05        2.1907
     -0.8517  -17115.7779  1.953998e+06
          Bot          Bot_Pattern_2   6647.3528    5.5521   6.858178e+06        4.8251        0.5541  119864.2013   -3060.1230  -7.612544e+05       59.2534
     96.7790   -9389.7597 -1.063558e+06
          Bot          Bot_Pattern_3  27623.2122    6.1652  -7.503240e+06        7.4086       -4.1771   10002.1801   15808.9696   2.068328e+05       79.1250
    117.2760   10718.7456  6.481819e+06
          Bot          Bot_Pattern_4  11148.8541    5.3857   2.328344e+06        1.2939        1.3868 -128585.2711    9851.2278  -9.389313e+05        2.7501
     -1.0597  -19149.2924  2.066642e+06
          Bot          Bot_Pattern_5  19465.2792    7.0869  -5.472319e+06       -0.4659        5.2906  -79087.7237   -4019.3052   2.253076e+05       -9.3003
      3.5916   13984.6563   6.318067e+06
 Infiltration Infiltration_Pattern_1  23351.2574    6.2459   1.899316e+06        2.2970        2.7252   52798.1665   40884.3830   1.880669e+06        1.1488
     14.3396  642195.9158  5.909028e+06
 Infiltration Infiltration_Pattern_2   3949.9532    6.1637   2.148014e+07       11.6336        8.4959   15655.3102   -4178.7052   1.915064e+06      213.4403
```

Figure 5.10. Reconstruction of Subpattern Centers in the Original Feature Space

As shown in Figure 5.10, the interpretation module loaded the PCA configuration and the family-specific K-Means models before reconstructing the cluster centers in the original feature space. The process covered 22 behavioral subpatterns distributed across the four traffic families: two Benign subpatterns, five Bot subpatterns, nine Infiltration subpatterns, and six Web Attack subpatterns.

The reconstructed centers provide representative values for several original traffic characteristics, including destination port, protocol, flow duration, forward and backward packet counts, byte and packet rates, flow inter-arrival time, packet-length statistics, active time, and idle time. These values provide a more interpretable description of the behavioral characteristics associated with each generated subpattern.

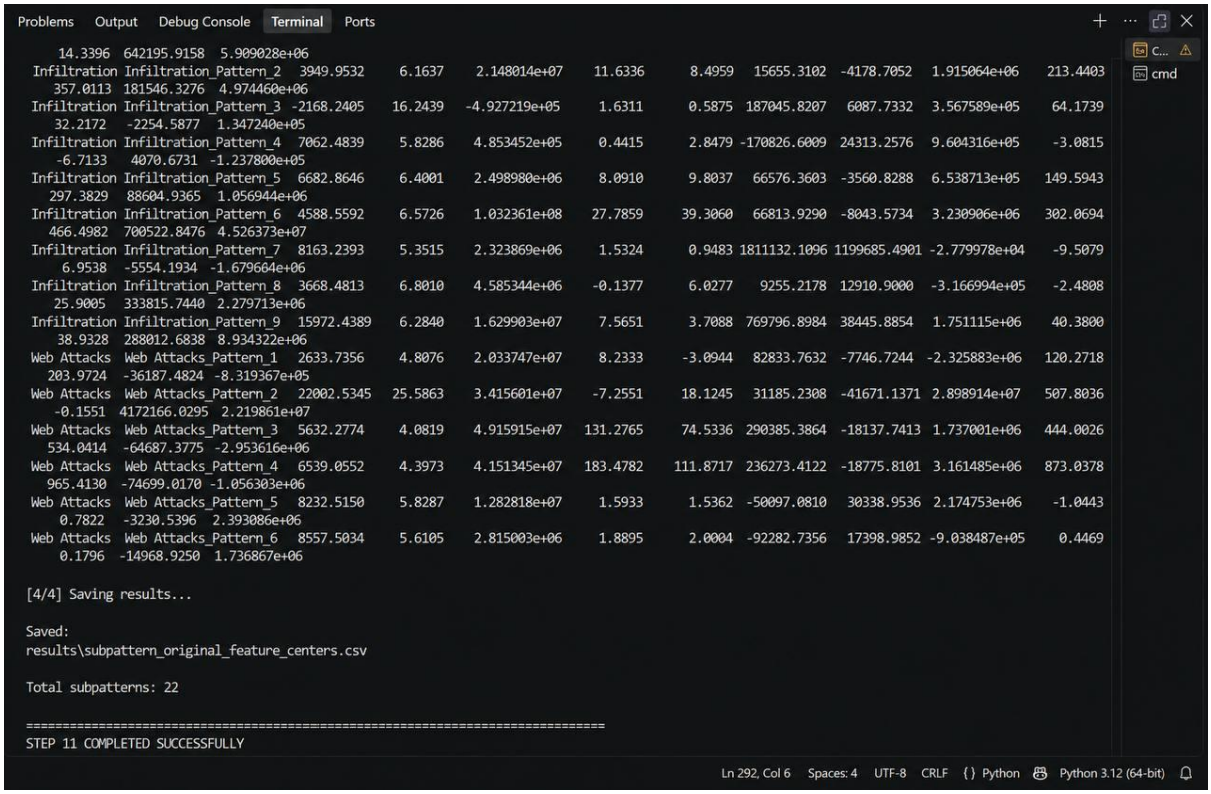


Figure 5.11. Completed Original-Feature Interpretation of the Generated Subpatterns

Figure 5.11 presents the remaining reconstructed subpattern centers and confirms successful completion of the interpretation stage. The resulting feature-center information was stored in the file subpattern_original_feature_centers.csv, and the final output confirmed that all 22 generated subpatterns were successfully processed.

This interpretation stage complements the classification results by providing a behavioral description of the generated subpatterns in terms of recognizable network features. Rather than treating the subpatterns only as cluster identifiers, the reconstructed feature centers make it possible to examine how the underlying traffic characteristics differ among the generated behavioral groups.

Overall, the subpattern-interpretation results provide an additional analytical layer for understanding the traffic behaviors identified by the Artificial Intelligence pipeline. This information supports the interpretation of the generated attack patterns before the resulting detection information is used by the subsequent Game Theory decision stage.

5.7 Game Theory Decision Results

After the AI detection results were transferred to the decision layer, the Game Theory Engine evaluated the eligible traffic samples and selected an appropriate defense strategy for each case. The decision process used the detected attack information and joint confidence to estimate the probability distribution of the possible attacker states. These probabilities were

then combined with the defender payoff model to calculate the expected utility of each available defense strategy and select the strategy with the highest expected utility.

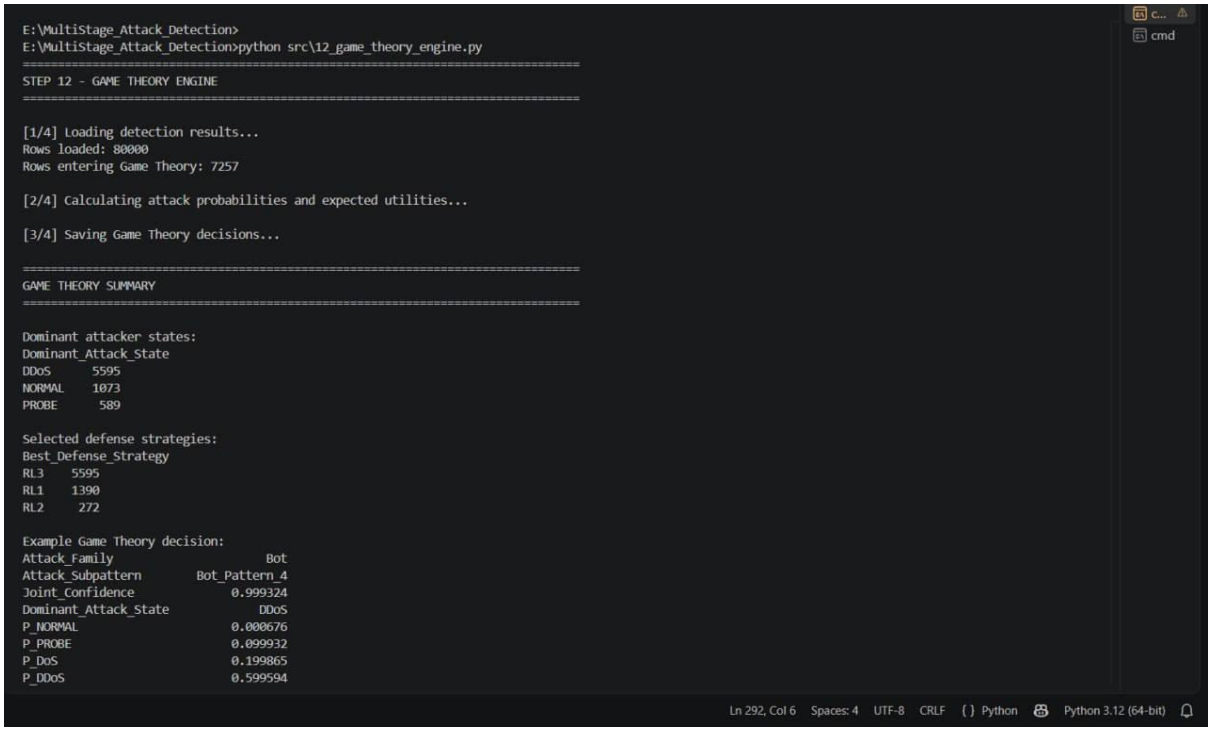


Figure 5.12. Game Theory Decision Summary and Selected Defense Strategies

As shown in Figure 5.12, the Game Theory Engine loaded 80,000 detection records, of which 7,257 satisfied the conditions required to enter the Game Theory decision stage. This confirms that the decision engine was applied selectively to the traffic records forwarded by the preceding AI-to-Game-Theory integration stage rather than processing all traffic samples as potential attacks.

The dominant-state analysis classified 5,595 of the evaluated records as DDoS, 1,073 as NORMAL, and 589 as PROBE. DDoS therefore represented the dominant attacker state for the majority of the records processed by the decision engine. The presence of different dominant states demonstrates that the engine did not apply a single fixed interpretation to all forwarded samples, but instead evaluated each record according to its estimated attack-state probability distribution.

The selected defense strategies also reflected the severity of the estimated attacker states. RL3 was selected for 5,595 records, RL1 for 1,390 records, and RL2 for 272 records. The predominance of RL3 indicates that the decision engine frequently selected a strong rate-limiting response for the traffic evaluated in this experiment, while less restrictive rate-limiting strategies were selected for the remaining cases.

Figure 5.13. Example Game-Theoretic Decision Based on Attack Probabilities and Expected Utilities

Figure 5.13 presents a detailed example of an individual Game Theory decision. The analyzed record was identified by the AI subsystem as Bot_Pattern_4 within the Bot attack family, with a joint confidence of approximately 0.9993. Based on this detection information, the Game Theory Engine generated probabilities for the possible attacker states and identified DDoS as the dominant state with a probability of approximately 0.5996.

For this example, the estimated state probabilities were 0.000676 for NORMAL, 0.099932 for PROBE, 0.199865 for DoS, 0.599594 for DDoS, and 0.099932 for Spoofed_DDoS. The probability distribution therefore assigned the greatest weight to the DDoS state while retaining lower probabilities for alternative attack states.

The engine subsequently calculated the expected utility of each available defense strategy. In the presented example, the expected utilities were -6.588774 for ALLOW, 0.903449 for RL1, 5.097904 for RL2, 6.794725 for RL3, and 5.49087 for BLOCK. Since RL3 produced the highest expected utility, it was selected as the optimal defense strategy for this particular decision.

Although BLOCK also produced a positive expected utility, its value was lower than that of RL3 in this example. Consequently, the decision mechanism selected strong rate limiting rather than complete blocking. This result illustrates the adaptive nature of the proposed defense approach, in which the selected response is determined by the calculated utility rather than by automatically applying the most restrictive action.

The generated Game Theory decisions were successfully saved in both CSV and JSON formats, and the completion message shown in Figure 5.13 confirms that the full decision

stage was executed successfully. These outputs provide the input required for the subsequent SDN action-mapping stage.

Overall, the results demonstrate that the Game Theory Engine successfully transformed AI-derived attack information into adaptive defense decisions. By estimating attacker-state probabilities, calculating the expected utility of multiple defensive strategies, and selecting the strategy with the highest utility, the framework provides a systematic transition from attack detection to response selection.

5.8 SDN Action Mapping Results

Following the Game Theory decision stage, the selected defense strategies were translated into executable SDN-oriented actions. The SDN Action Mapper converted each Game Theory strategy into the corresponding action type, rate limit, and OpenFlow action required for subsequent enforcement.

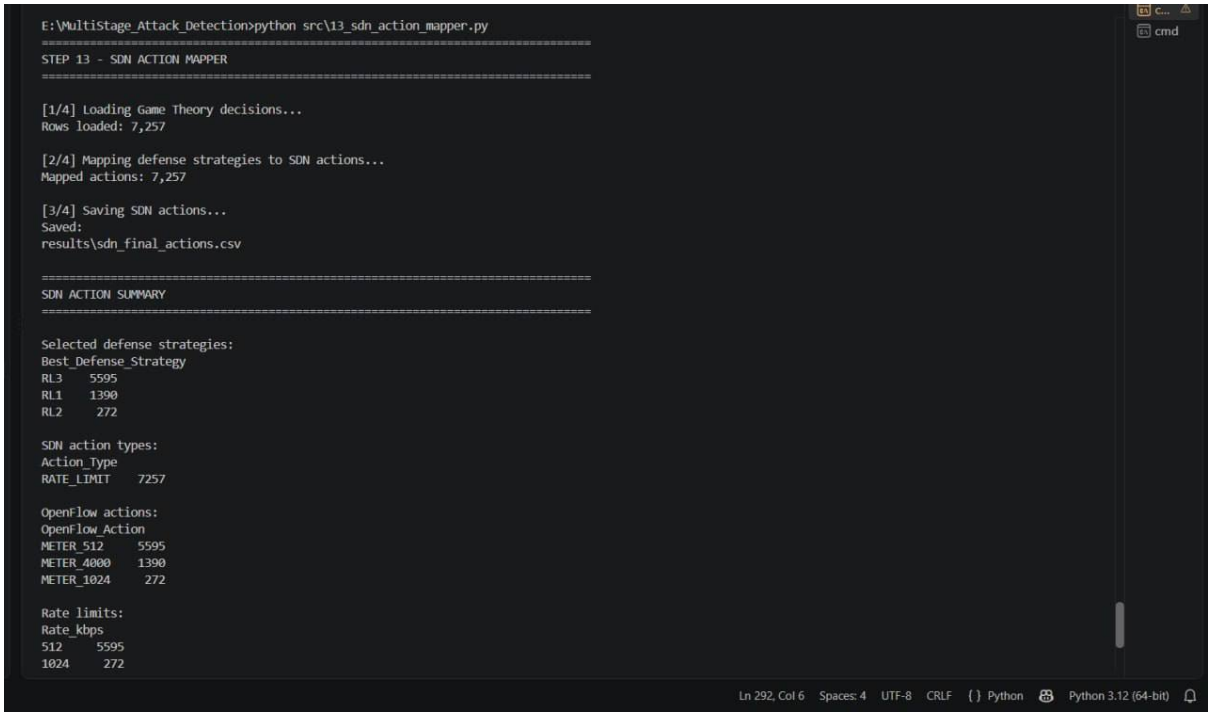


Figure 5.14. SDN Action Mapping Summary

As shown in Figure 5.14, all 7,257 Game Theory decisions were successfully mapped to SDN actions. The selected RL1, RL2, and RL3 strategies were translated into the corresponding OpenFlow meter actions, including METER_4000, METER_1024, and METER_512, respectively.

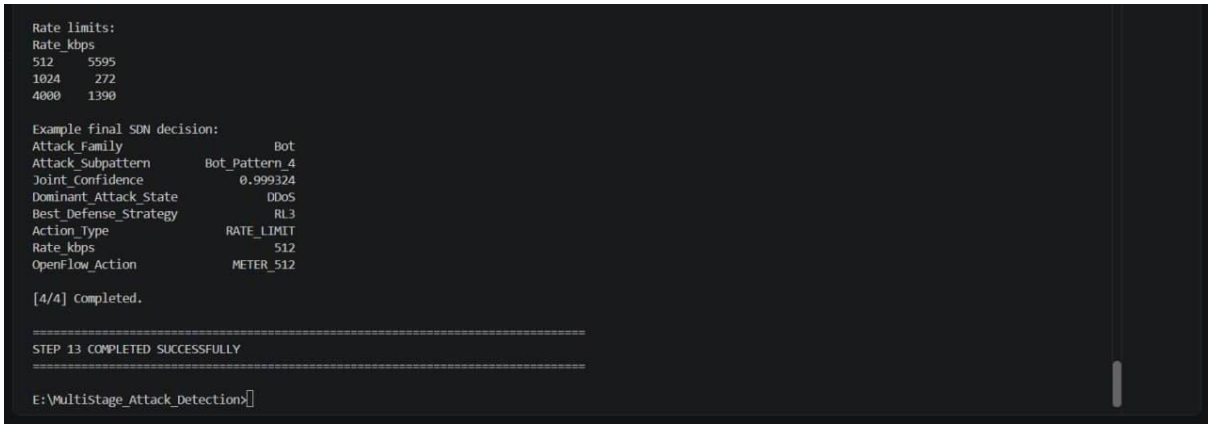


Figure 5.15. Example Final SDN Action Generated from the Game Theory Decision

Figure 5.15 illustrates the complete mapping of an individual defense decision. For the presented Bot_Pattern_4 example, the RL3 strategy selected by the Game Theory Engine was converted into a RATE_LIMIT action with a rate of 512 kbps and the OpenFlow action METER_512. This result confirms the successful transition from strategic defense selection to an SDN-compatible mitigation action.

5.9 OpenFlow Rule Generation Results

After the SDN action-mapping stage, the resulting defense actions were converted into OpenFlow-compatible meter and flow-rule templates. This stage prepared the selected rate-limiting strategies for enforcement through Open vSwitch using OpenFlow 1.3 commands.

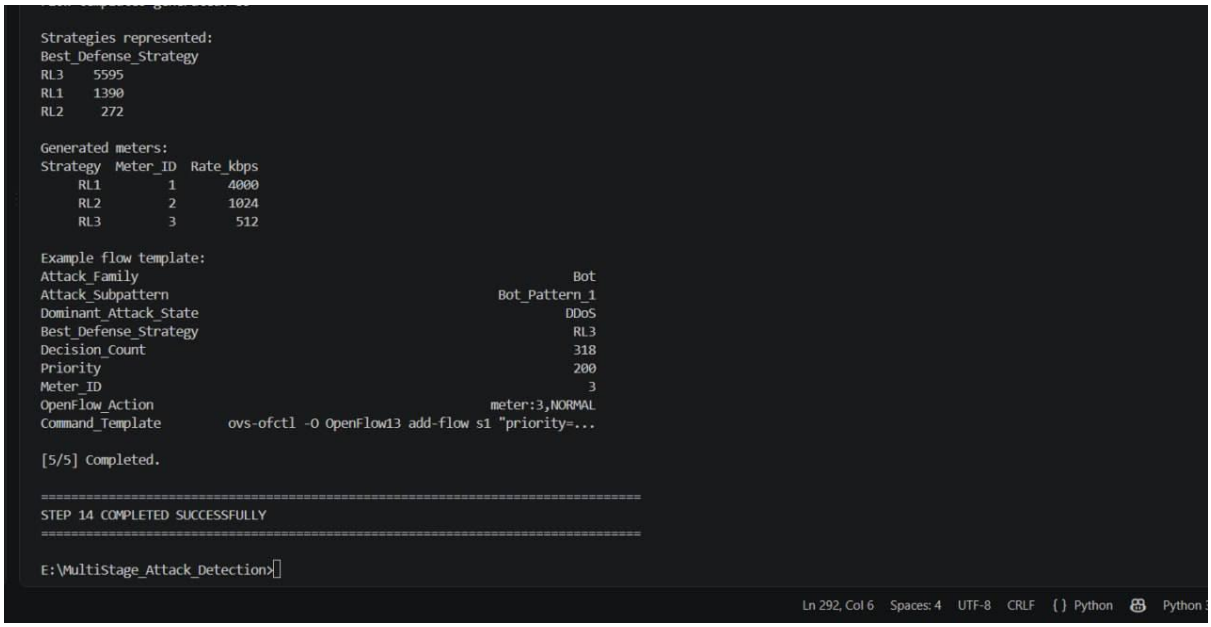


Figure 5.16. Generated OpenFlow Meter and Flow-Rule Template

As shown in Figure 5.16, the three rate-limiting strategies were mapped to dedicated OpenFlow meters: RL1 to 4000 kbps, RL2 to 1024 kbps, and RL3 to 512 kbps. The generated flow template also associated the selected defense strategy with the corresponding meter identifier, priority, OpenFlow action, and command template.

The successful completion of Step 14 confirms that the defense decisions produced by the previous stages were translated into OpenFlow-compatible rule templates, completing the transition from AI-based detection and Game Theory decision making to SDN-oriented mitigation preparation.

5.10 Chapter Summary

This chapter presented the experimental evaluation of the enhanced security framework. The results demonstrated the progression from multi-stage attack detection to AI-based hierarchical classification, weak-subpattern analysis, subpattern interpretation, Game Theory decision making, SDN action mapping, and OpenFlow rule generation. The experimental outputs confirmed that the proposed framework successfully integrates detailed attack identification with adaptive defense selection and SDN-oriented mitigation.

6 Chapter Six: Conclusion

This project presented the development and enhancement of an adaptive security framework for Software-Defined Networking environments. The original framework established the fundamental defense architecture by integrating Mininet, Open vSwitch, the Ryu Controller, Snort IDS, Game Theory, and OpenFlow-based mitigation. This provided a closed-loop mechanism in which suspicious network activity could be observed, evaluated, and followed by an appropriate defensive response.

The framework was subsequently extended to support multi-stage attack analysis. Instead of evaluating suspicious traffic only as isolated events, the enhanced controller maintains information about previous and current attack states and uses stage-transition information to represent the evolution of malicious behavior across consecutive observation rounds. This extension provides additional temporal context for the Game Theory decision process and enables the defense mechanism to respond to changing attack behavior more adaptively.

A second major extension introduced an Artificial Intelligence-based detection pipeline for more detailed analysis of complex network traffic. The pipeline included data preparation, feature engineering, dimensionality reduction using PCA, attack-pattern segmentation, and hierarchical classification. The final hierarchical CatBoost architecture identifies both the general traffic family and the corresponding behavioral subpattern while retaining confidence information for the generated prediction.

The experimental evaluation demonstrated strong performance for the hierarchical classification model. On the test dataset, family-level accuracy reached 98.05%, while subpattern-level accuracy reached 97.28%. The test macro F1-score reached 93.44%, and the weighted F1-score reached 97.81%. In addition, only 5.10% of the test predictions were identified as low-confidence cases. These results indicate that the developed model can provide detailed attack identification while maintaining high overall classification performance.

The additional error and subpattern analyses provided further insight into the behavior of the classifier. Weak subpatterns were identified to determine where classification remained difficult, while the interpretation stage reconstructed subpattern centers in the original network-feature space. These analyses improved the interpretability of the AI component by showing that classification errors were concentrated in specific behavioral patterns rather than representing a general weakness of the model.

The final contribution of the enhanced framework was the integration of the AI detection output with the existing SDN defense architecture. Attack-family, subpattern, and confidence information was successfully transferred from the Windows-based AI environment to the VMware-based Mininet environment. The Game Theory Engine then converted this detection information into attacker-state probabilities, evaluated the expected utilities of the available defense strategies, and selected the most appropriate response. The selected strategy was subsequently mapped to an SDN action and translated into OpenFlow-compatible meter and flow-rule templates.

Overall, the enhanced framework establishes an end-to-end transition from network observation and multi-stage attack tracking to detailed AI-based identification, strategic defense selection, and SDN-oriented mitigation. The main contribution of the project therefore lies not in a single detection or mitigation technique, but in the integration of these components into a coordinated security workflow that combines temporal attack awareness, hierarchical Artificial Intelligence, Game Theory, and programmable network control.

The developed framework remains a proof-of-concept implemented primarily in controlled and emulated environments.
The SDN experiments were conducted using virtualized Mininet and VMware components, while the Artificial Intelligence models were evaluated using prepared network-traffic datasets rather than continuous production traffic. Furthermore, although the hierarchical model achieved high overall performance, the weak-subpattern analysis showed that some detailed attack behaviors remain more difficult to distinguish than others. Therefore, additional validation would be required before deploying the complete framework in a large-scale operational network.

Future work can extend the framework by evaluating it on larger SDN topologies, additional attack scenarios, and real-time network traffic. Further experiments may examine controller overhead, end-to-end mitigation latency, scalability, false-positive behavior, and the effect of defensive actions on legitimate traffic. The Artificial Intelligence component can also be extended using additional datasets and improved methods for difficult or overlapping subpatterns. In addition, the Game Theory payoff model may be refined using experimentally derived measurements or historical security data, while the integration layer can be evaluated under continuous real-time communication between the AI, controller, and OpenFlow enforcement components.

In conclusion, this project demonstrates the feasibility of extending an SDN and Game Theory-based security framework with multi-stage attack awareness and Artificial Intelligence-based hierarchical detection. By connecting detailed attack identification with adaptive decision making and programmable SDN mitigation, the developed framework provides a unified experimental foundation for studying intelligent and adaptive network defense.

List of Terms and Abbreviations

Full Term	Abbreviation
Software-Defined Networking	SDN
Distributed Denial of Service	DDoS
Denial of Service	DoS
Intrusion Detection System	IDS
Virtual Machine	VM
Internet Control Message Protocol	ICMP
Internet Protocol	IP
Transmission Control Protocol	TCP
User Datagram Protocol	UDP
Round Trip Time	RTT
Packets Per Second	pps
Kilobits Per Second	kbps
Quality of Service	QoS
Central Processing Unit	CPU
Long Term Support	LTS
Rate Limiting Level 1	RL_1
Rate Limiting Level 2	RL_2
Rate Limiting Level 3	RL_3
Economic Denial of Sustainability	EDoS
Artificial Intelligence	AI
Principal Component Analysis	PCA
Principal Component	PC
Comma-Separated Values	CSV
JavaScript Object Notation	JSON
Hypertext Transfer Protocol	HTTP

Full Term	Abbreviation
Secure Shell	SSH
Support Vector Machine	SVM
F1-Score	F1

References

[1] F. Razvan and C. Mitica, “Enhancing network security through integration of game theory in software-defined networking framework,” International Journal of Information Security, vol. 24, art. no. 100, 2025, doi: 10.1007/s10207-025-01012-4.

[2] N. Gupta, S. Tanwar, and S. Badotra, “A novel sFlow based DDoS detection model in software defined networking,” International Journal of Applied Science and Engineering, vol. 21, no. 2, art. no. 2023510, 2024, doi: 10.6703/IJASE.202406_21(2).005.

[3] R. Wainwright, M. Bagheri, A. Salama, and R. Saatchi, “Software-Defined Networking Security Detection Strategies and Their Limitations with a Focus on Distributed Denial-of-Service for Small to Medium-Sized Enterprises,” Applied Sciences, vol. 15, art. no. 12389, 2025, doi: 10.3390/app152312389.

[4] A. Kachavimath and N. D. G., “Hybrid Approach for Detection and Mitigation of DDoS Attacks Using Multi-feature Selection, Unsupervised Learning, and Game Theory,” Journal of Telecommunications and Information Technology, no. 4, pp. 20–32, 2025, doi: 10.26636/jtit.2025.4.2261.

[5] M. M. Trigui, M. S. Islam, M. D. Gambo, M. S. Alam, and T. Helmy, “Resource-Aware Ensemble Framework for DDoS Detection in SDN via Novel Game-Theoretic Feature Selection,” IEEE Access, vol. 14, pp. 1666–1676, 2026, doi: 10.1109/ACCESS.2025.3618519.

[6] A. Javadpour, F. Ja’fari, T. Taleb, and C. Benzaïd, “Detecting malicious nodes using game theory and reinforcement learning in software-defined networks,” International Journal of Information Security, vol. 24, art. no. 117, 2025, doi: 10.1007/s10207-025-01026-y.

[7] F. Z. Chowdhury, M. N. Adnan, M. M. Siddique, A. Parvez, and M. Y. I. B. Idris, “Advanced EDoS eye: A pragmatic model to mitigate economic denial of sustainability attack in cloud system applying dynamic game-based decision module,” International Journal of Innovative Research and Scientific Studies, vol. 9, no. 2, pp. 35–52, 2026, doi: 10.53894/ijirss.v9i2.11243.

[8] S. Gong, J. Cho, and K. K. Choi, “Detection of Multi-Stage Attacks Through Attack Pattern Segmentation,” IEEE Access, vol. 13, pp. 204155–204167, 2025, doi: 10.1109/ACCESS.2025.3635053.

[9] R. Kour, R. Karim, and P. Dersin, “Modelling cybersecurity strategies with game theory and cyber kill chain,” International Journal of System Assurance Engineering and Management, 2025, doi: 10.1007/s13198-025-02733-4.